Unit 1
Introduction to Analysis of Algorithm

Dr. Anupama Phakatkar

CE Department Vision & Mission

Vision

- The Department of Computer Engineering strives to be a center of academic excellence in computer engineering education, fostering multidisciplinary research and empowering individuals to embrace global opportunities.

- **Mission**

1. Provide strong conceptual knowledge and hands-on experience to empower students for developing solutions by applying design thinking using state-of-the-art tools and technologies to shape individuals into successful professionals.

2. Foster a dynamic research and innovation ecosystem that drives multidisciplinary advancements in computing.

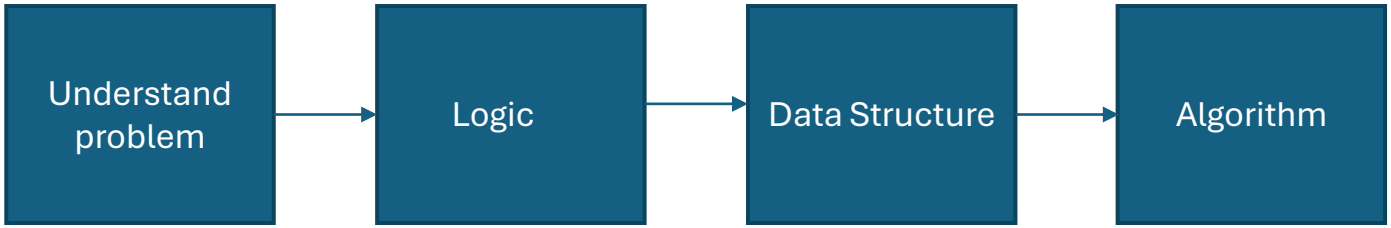
3. Inculcate professional ethics and effective communication skills to nurture leadership potential to address challenges in the field of Computer Engineering.

Importance of Data Structure

- Help computer and human to understand and work with data more efficiently
- Used to store, organize and manipulate data and part of CS algorithm
- Improved code efficiency
- Manage large amount of data
- Improve program performance
- Solve complex problems
- Understand data relationships
- Frame information organization
- Applications:
  - Social networks
  - Navigation system
  - E-commerce

From Problem to Solution

- Steps followed to solve the problem



- Example :- Calculate average of ‘N’ numbers
- Problem :- Formula of averaging
- Logic :- Need to know sum of all numbers, how many numbers
- Data Structure :- Array
- Algorithm :-
  - Loop for adding all number
  - Divide sum by ‘N’

Basics

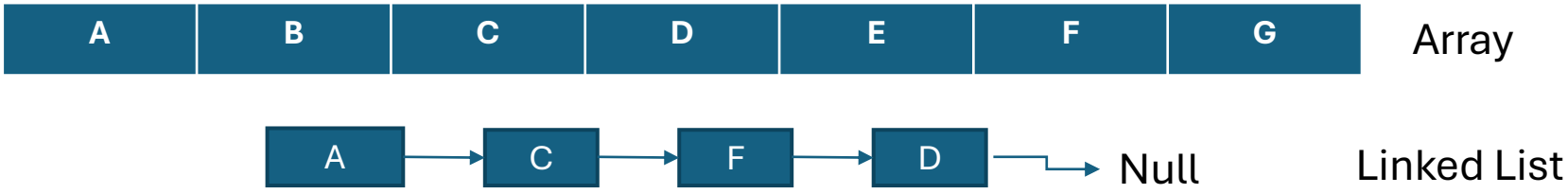
- Data :- It is the collection of facts. It is description of the things.
- Information :- The valuable things derived from data is information.
- Knowledge :- Use information to achieve our goal is knowledge.
- Example
  - Data:- employee names, salary, date of birth.
  - Information:- If we integrate all field we will get information.
  - Knowledge :- The integrated information becomes act as knowledge to achieve the our goal.
  - List the employees having salary > “Value”
  - List the employees born after 1-1-2001

Data Structure

- A data structure is a data organization, management, and storage format that enables efficient access and modification.
- It is a collection of data values, the relationships among them, and the functions or operations that can be applied to the data.
- Example:- An ARRAY is a Data structure stores number of elements of the same type in a specific order.
- Data values:- All values of same data type
- Relationship:- specific order of elements
- Operations:- store at any location, retrieve from given location, display

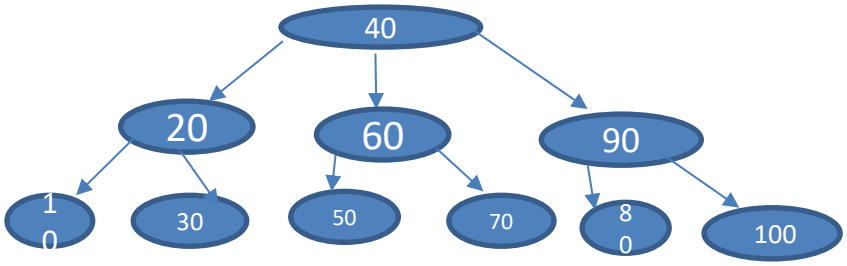
Types of Data Structures

- **Linear Data Structure**
  - In linear data structures, the elements are arranged in sequence one after the other.
  - Every data element has a unique successor and predecessor.
    - Two basic ways of representing linear structures in memory.
    - Since elements are arranged in a particular order, they are easy to implement.
  - One way is to have the relationship between the elements by means of pointers (links), called linked lists.
  - The other way is using sequential organization, that is, arrays.

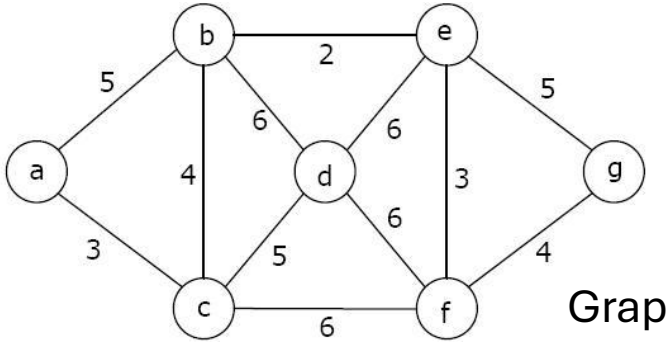


Types of Data Structures

- **Non-Linear Data Structure**
  - Elements in non-linear data structures are not in any sequence.
  - They are arranged in a hierarchical manner where one element will be connected to one or more elements.
  - Every data element may have more than one predecessor as well as successor.
  - Used to represent the data containing hierarchical or network relationships among the elements.
  - Trees and graphs are examples of non-linear data structures.



Tree



Graph

Types of Data Structures

- Based on Memory Allocation
  - Static and Dynamic Data Structures

- **Static Data Structures**
  - Memory size is fixed at compile time and cannot be changed during execution.
  - Stored in contiguous memory blocks.
  - Example: Static Array.
  - Advantages: Fast indexing and easy implementation.
  - Disadvantages: Memory may be wasted or insufficient if the size is estimated incorrectly.

Types of Data Structures

- **Dynamic Data Structures**
  - Memory size can grow or shrink at runtime depending on requirements.
  - Memory is allocated from the heap, often non-contiguous
  - Examples: Linked Lists, Dynamic Arrays, Trees
  - Advantages: Efficient memory usage and flexibility
  - Disadvantages: Slower due to pointer overhead and dynamic allocation.

Types of Data Structures

- Based on Life of Data structures
  - Persistent and Emphemeral Data Structures

- **Persistent Data Structures**
  - After an update, the old version is preserved, and a new version is created.
  - No modification happens in place (structure is immutable).
  - Examples: Persistent trees, immutable linked lists used in functional languages.
  - Applications: Undo/redo systems, version control (Git), analytics requiring past states

Types of Data Structures

- **Ephemeral Data Structures**
  - Updates occur in place, and old versions are lost after modification.
  - Most commonly used structures in everyday programming.
  - Examples: Normal arrays, linked lists, stacks, queues.
  - Uses: Fast updates, low memory consumption, suitable for general programs.

Summary

- Array :- Linear as well as static data structure
- Linked list :- Linear and dynamic data structure
- Stack, Queue :- Linear and Static or
  Linear and dynamic
- Tree, Graph :- Non-Linear and Static or
  Non-Linear and dynamic

Algorithm

- The word algorithm comes from the name of the person author Abu Jafar Mohammed Ibn Musa Al khowarizmi who wrote a textbook entitled “Algorithmi de numero indorum”

- An algorithm is an effective method for finding out the solution for a given problem

- An Algorithm is a finite set of instructions that if followed, accomplishes a particular task

OR

- An Algorithm is a step-by-step plan for a computational procedure that possibly begins with an input and yields an output value in a finite number of steps in order to solve a particular problem.

Algorithm

- An algorithm is an efficient method that can be expressed within finite amount of Time and Space.

- The important aspects of algorithm design include creating an efficient algorithm to solve a problem in an efficient way using minimum time and space.

- To solve a problem, different approaches can be followed. Some of them can be efficient with respect to time consumption, whereas other approaches may be memory efficient.

Algorithm Specification

To evaluate an algorithm, we have to satisfy the following criteria:

1. INPUT: The Algorithm should be given zero or more input.

2. OUTPUT: At least one quantity is produced.

3. DEFINITENESS: Each instruction is clear and unambiguous.

4. FINITENESS: If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after a finite number of steps.

5. EFFECTIVENESS: Every instruction must very basic so that it can be carried out, in principle, by a person using only pencil & paper.

Properties of Algorithm

Algorithm can be described (Represent) in four ways.

1. Natural language like English
2. Graphic representation called flowchart: This method will work well when the algorithm is small & simple.
3. Pseudo-code Method: In this method, we should typically describe algorithms as program, which resembles language like Pascal & Algol(Algorithmic Language). It is a notation resembling a simplified programming language, used in program design.
4. Programming Language: We have to use programming language to write algorithms like C, C++,JAVA etc.

How To Write an Algorithm

Step-1:start

Step-2:Read a,b,c

Step-3:if a>b

if a>c

print a is largest

else

if b>c

print b is largest

else

print c is largest

Step-4 : stop

Step-1: start

Step-2: Read a,b,c

Step-3:if a>b then go to step 4

otherwise go to step 5

Step-4:if a>c then

print a is largest otherwise

print c is largest

Step-5: if b>c then

print b is largest otherwise

print c is largest

step-6: stop

Differences

Algorithm

1. At design phase
2. Natural language
3. Person should have Domain knowledge
4. Analyze

Program

1. At Implementation phase
2. written in any programming language
3. Programmer
4. Testing

PERFORMANCE ANALYSIS

- An algorithm is said to be efficient and fast if it take less time to execute and consumes less memory space at run time is called Performance Analysis.
- Performance Measures:- Space Complexity & Time Complexity
- Space complexity is the amount of memory required to run to completion.
- Space Complexity is space needed by program is the sum of **fixed space requirement** and **variable space requirement**
- Fixed Space requirement is fixed part that is independent of the characteristics (e.g. number, size) of the inputs and outputs

PERFORMANCE ANALYSIS cont…

Fixed Space requirement includes

- Instruction space (space to store code)
- Simple variables
- Fixed size variables (structures, array)
- Constants

Variable space requirements

- Space required by component variables whose size depends on the particular instant I,
- Space required by referenced variables
- Space required for recursion (stack space)

The space requirement S(P) of any program P is written as

$$S(P) = c + S_p(\text{instance characteristics})$$

Where c is constant

Examples

Iterative function of sum

int sum(int *a, const int n)
{
int s =0;
for(int i=0;i<n;i++)
s+= a[i];
return(s);
}

Space required for function
2 bytes required to store a[0] or *a
2 bytes required to store s, i, n
Total = 8 bytes
$S_{sum}(4)$ = 8 bytes
$S_{sum}(100)$ = 8 bytes $S_{sum}(400)$ = 8 bytes $S_{sum}(n)$ = 0 Space complexity of sum function is constant or O(1)

Examples : Factorial

Iterative function of Factorial
1. int fact(int x) 2. { 3. int ans =1; 4. while(x>1) 5. ans= ans*x; 6. x = x-1; 7. Return(ans); 8. }

Space required for function
2 bytes required to store x
2 bytes required to store ans
Total = 4 bytes
$S_{sum}(4)$ = 4 bytes
$S_{sum}(100)$ = 4 bytes $S_{sum}(400)$ = 4 bytes $S_{sum}(n)$ = 0 Space complexity of sum function is constant or O(1)

Examples

Recursive function of sum
1. int Rsum(int *a, int n) 2. { 3. if(n<=0) return 0; 4. else 5. return(Rsum(a,n-1) + a[n-1]); 6. }

Space required for function
Rsum instances will be = n+1
For every instance stack space includes formal parameters, local parameters, return address
Rsum requires 8 bytes (2 bytes *4) per call [*a, n] – formal parameters, 1- return value, 1- return address
Total memory required $S_{Rsum}(n) = 8(n+1)$
$S_{Rsum}(2) = 8(n+1) = 24$ bytes
$S_{Rsum}(50) = 8(n+1) = 408$ bytes
Space complexity of Rsum function is proportional to n or O(n)

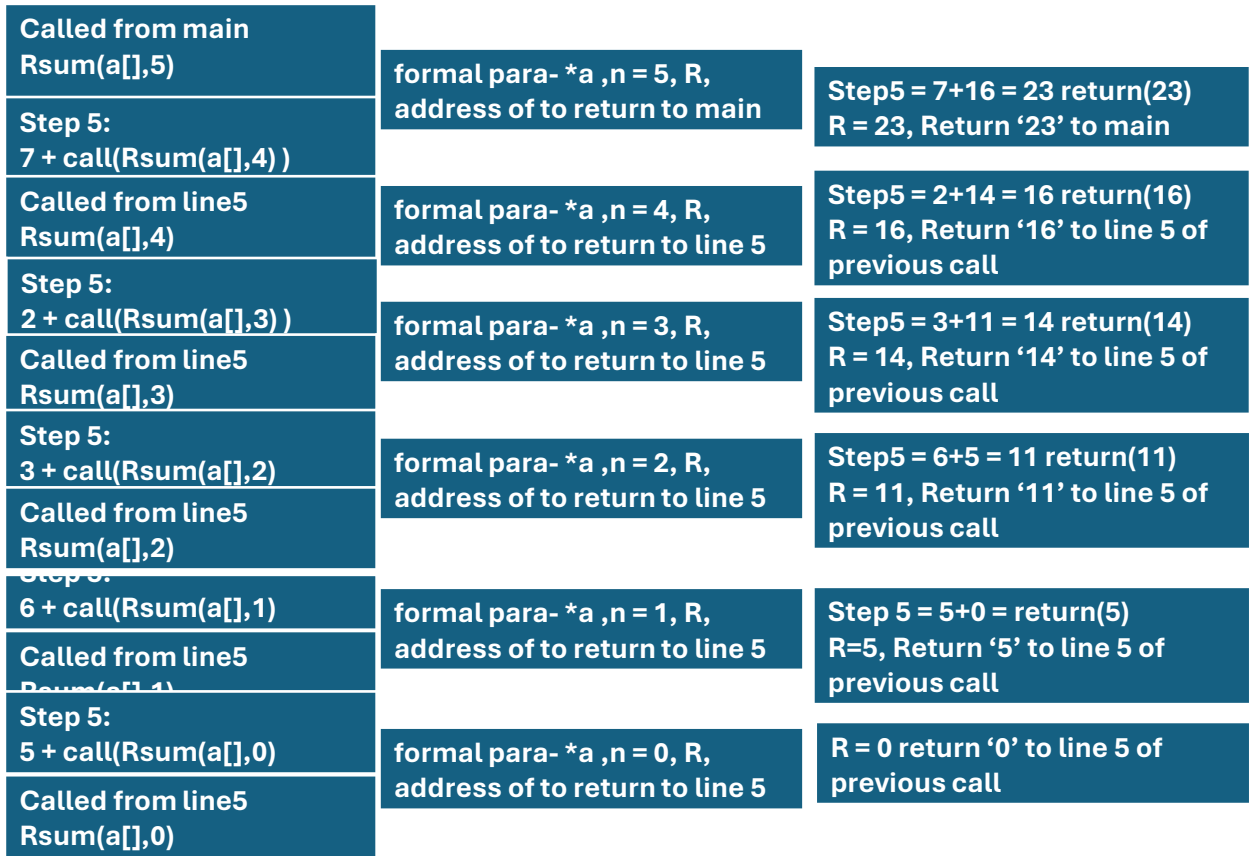
Recursion

Recursive function of sum

```
1. int Rsum(int *a, const int n)
2. {
3.     if(n==0) return 0;
4.  else
5.       return(Rsum(a,n-1) + a[n-1]);
6. }
```

Sum= Rsum(a,5)

0	1	2	3	4
5	6	3	2	7



Examples

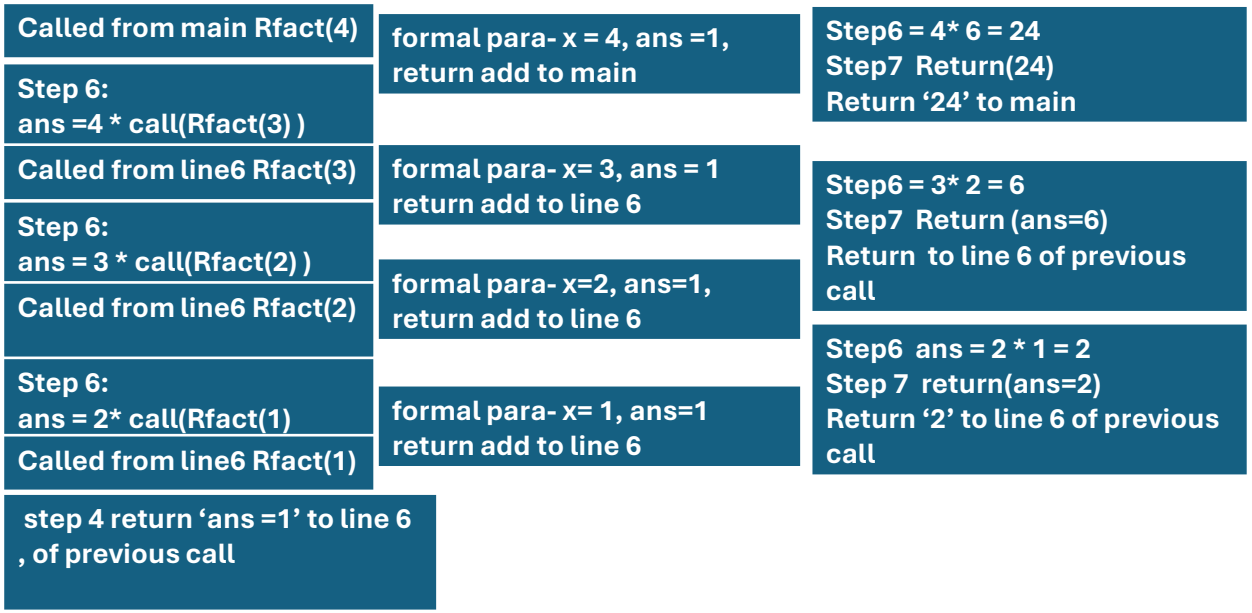
Recursive function of factorial
1. int Rfact(int x) 2. { 3. if(x ==1) 4. return 1; 5. else 6. return(x * Rfact(x-1)); 7. }

Space required for function
Rfact instances will be = n
For every instance stack space includes formal parameters, local parameters, return address
Rfact requires 6 bytes (2bytes *3[x, return value, return address]) per call
Total memory required $S_{Rsum}(n) = 6(n)$
$S_{Rsum}(2) = 6(n) = 12$ bytes
$S_{Rsum}(50) = 6(n) = 300$ bytes
Space complexity of Rfact function is proportional to n or O(n)

Recursion

Recursive function of factorial

```
1.  int Rfact(int x)
2.  { int  ans = 1;
3.   if(x ==1)
4.        return ans;
5.   else
6.        ans= x* Rfact(x-1)
7.        return(ans);
8.  }
F =  Rfact(4)
```



Time Complexity

- The time, T(P), taken by a program P is sum of the compile time and the run time.
- We can run the program without recompilation
- In time complexity only run time is considered for calculations
- Step count method :-
  - Calculate step count of a program by listing total number of steps contributed by each statement
  - To calculate step count
    - Find step per execution (s/e) for each statement
    - Identify frequency of each statement
    - Total steps = s/e * frequency

Step/Execution

Statement	Details	S/e
Comment	Comments are not executable statements	0
Declarative statements	Define variables, constants, user defined datatypes, determine access, characterize functions (void, virtual)	0
Expressions, assignment	Var = expr	1
Iteration	For(init; expr; increment) While(expr) Do ..while(expr)	Init -1, expr -1, increment-1/ 1-for complete for statement
Switch statement	Switch condition Case 1: Case 2 : Default: }	The s/e of each condition is its cost plus that of all preceding conditions (1s/e for each condition)

Step/Execution

Statement	Details	S/e
Conditional statements	If expr s1; Else s2;	Each part is assigned the number of steps corresponding to expr, s1,s2
Function Invocation	All invocations count as 1 step unless the invocation involves pass-by-value parameters	1 for function without parameters n for function with n parameters pass by value
Memory management	New, delete, sizeof. If Invoking constructor/destructor then treated as Function invocation	1
Jump statement	Continue,break,return	1 if return(expr) then s/e of expression

Examples

Iterative function of sum	S/e	frequency	Total step
1. int sum(int *a, const int n)			
2. {			
3. int s =0;	1	1	1
4. for(int i=0;i<n;i++)	1	n+1	n+1
5. s+= a[i];	1	n	n
6. return(s);	1	1	1
7. }			
			Total = 2n+3

Examples

Iterative function of sum	S/e	frequency	Total step
int sum(int *a, const int n)			
1. {			
2. int s =0;	1	1	1
3. for(int i=0;i<n;i++)			
3.1 i=0	1	1	1
3.2 i<n	1	n+1	n+1
3. 3 i++	1	n	n
4. s+= a[i];	1	n	n
5. Return(s);	1	1	1
6. }			Total = 3n+4

Examples

Recursive function of sum	s/e	Frequency		Total steps	
		N=0	N>0	N=0	N>0
int Rsum(int *a, const int n)					
1. if(n<=0)	1	1	1	1	1
2. return 0;	1	1	0	1	0
3. else					
4. return(Rsum(a,n-1) + a[n-1]);	$1+T_{Rsum}(n-1)$	0	1	0	$1+T_{Rsum}(n-1)$
6.					
				Total steps = $2+ T_{Rsum}(n-1)$	

F(n) = 2+F(n-1) n>0
=2 n<=0
F(4) = 2+f(3)
=2 + 2+f(2)
= 2+2+ 2+f(1)
=2 + 2+ 2+ 2+ f(0)
=2*n + 2
=2*(n+1)

Examples

Iterative function of factorial	S/e	frequency	Total step
int fact(int x) 1.{ 2. int ans =1; 3. while(x>1) 4. ans= ans*x; 5. x = x-1; 6. Return(ans); 7. }	 1 1 1 1 1	 1 X X-1 X-1 1	 1 X X-1 X-1 1 Total = 3X

Examples

Recursive function of factorial	s/e	Frequency		Total steps	
		x=1	x>1	x=1	x>1
int Rfact(int x) 1. { 2. if(x <=1) 3. return 1; 4. else 5. return(x * Rfact(x-1)); 6. }	1 1 $1+T_{Rsum}(x-1)$	1 1 0	1 0 1	1 1 0	1 0 $1+T_{Rsum}(x-1)$
				Total steps = $2+ T_{Rsum}(x-1)$	

F(n) = 2+f(n-1) = 2+2+f(n-1).....2*(n-1)+F(1)
F(3) = 2+f(2)
= 2+2+f(1)
=2+2+2
F(n) = 2+2(n-1) = 2(n)

Fibonacci Function

Iterative function of fibonacci	S/e	frequency	Total step
int fib(int x)			
1.{	0	0	0
2. int f0=0;	1	1	1
3. int f1 =1; int ans;	1	1	1
4. while(x>2)	1	X-1	X-1
4. {	0	0	0
5. ans= f0+f1;	1	X-2	X-2
5. f0 = f1;	1	X-2	X-2
6. f1 = ans;	1	X-2	X-2
7. x = x-1;	1	X-2	X-2
8. }	0	0	0
6. Return(ans);	1	1	1
7. }	0	0	0
			Total = 5X-6

Fibonacci function

Recursive function of fibonacci	S/e	frequency		Total step	
		X<=1	X>1	X<=1	X>1
int fib(int x)					
1.{	0	0	0	0	0
2. if(x<=1)	1	1	1	1	1
3. Return(x)	1	1	0	1	0
4. else	0	0	0	0	0
5. Return(fib(x-1) +fib(x-2))	$1+T_{fib}(X-1)+T_{fib}(X-2)$	0	1	0	$1+T_{fib}(X-1)+T_{fib}(X-2)$
6. }	0	0	0	Total = 2	Total = $2+T_{fib}(X-2)+T_{fib}(X-1)$

$T(n) = 2+ T(n-1) + T(n-2)$ let $T(n-1)=T(n-2)$

$= 2+ 2T(n-1)$

$T(n) = 2 + 2T(n-1)$

$= 2 + 2[2+ 2T(n-2)] = 2+4+...2^K(T(n-k))$

$=2 + 4 + 4T2 = 2 + 4 + 8 =$

$= 2^{(n-1)}$

$T(n) = 2+ T(n-1) + T(n-2)$ $T(n-1)=T(n-2)$

$= 2+ 2T(n-2)$

$T(6) = 2 + 2T(4) = 2 + 2 + 2T(2)$

$=2^{(n/2)}$

Sample examples to calculate Space and Time Complexity

Write pseudocode for following and find space and time complexity for the same

A) Convert given number from Binary and octal

B) Convert given number from Octal to Binary

C) Convert given octal number into Hex

D) Convert given Hex number into Octal

E) Convert given number decimal into Roman

F) Convert given number Roman number to decimal

G) Write a program to find compound interest for given information. Identify inputs required and then calculate compound interest.

H) Write a program for currency conversion.

I) Write a program to count vowels in given string.

J) Write a program to generate palindrome of given string.

K) Write a program to convert upper case to lower and vice versa characters in given string.

L) Write a program to encrypt and decrypt the string. Define your own rules and constraints.

M) Write a program to implement set operations on given sets.

Read/store all elements in 2D-array/Matrix

```
Void Read_all( int **A, int m, n)
1.    For i=0;i<m;i++
2.    For j=0;j<n;j++
3.          Print(A[i][j])
4.    End for
5.    End for
```

```
Step count
M+1
M(N+1)
MN
0
0
Total = 2M+2MN + 1
```

```
Void store_all(int **A int m)
1.    For i=0;i<m;i++
2.    For j=0;j<n;j++
3.          input(A[i][j])
4.    End for
5.    End for
```

```
Step count
M+1
M(N+1)
MN
0
0
Total = 2M+2MN+1
```

Addition of elements in 2 Matrix

```
Void Addition( int **A,int **B,
int m, n)
1.    int  c[m][n];
2.    For i=0;i<m;i++
3.    For j=0;j<n;j++
4.          c[i][j] = A[i][j] + b[i][j];
5.    End for
6.    End for
```

```
Step count


M+1
M(N+1)
MN
0
0
Total = 2M+2MN + 1
```

If M == N then
Total = $2M^2 + 2M + 1$

Complexity - $O(n^2)$

$$A + B = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} + \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \cdots & b_{mn} \end{bmatrix}$$

$$= \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} & \cdots & a_{1n} + b_{1n} \\ a_{21} + b_{21} & a_{22} + b_{22} & \cdots & a_{2n} + b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} + b_{m1} & a_{m2} + b_{m2} & \cdots & a_{mn} + b_{mn} \end{bmatrix}$$

Transpose of 2D Matrix

```
Void Transpose( int **A, int m,int n)
1.    int c[n][m];
2.    For i=0;i<m;i++
3.    For j=0;j<n;j++
4.          c[i][j] = A[j][i] ;
5.    End for
6.    End for
```

Step count
M+1
M(N+1)
MN
0
0
Total = 2M+2MN + 1

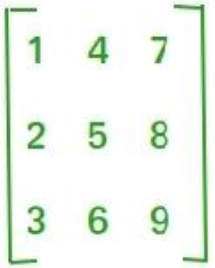
If M == N then

Total = $2M^2 + 2M + 1$

Complexity - $O(n^2)$



Input



Output

Multiplication of elements in 2 Matrix

```
Void Multiplication ( int **A,int **B,int m, int n, int p)
1.    int c[m][p];
2.    For i=0;i<m;i++
3.    For j=0;j<n;j++
4.    C[i][j] = 0
5.    For k=0;k<p;k++
6.          c[i][j] = c[i][j] + (A[i][k] * b[k][j]);
7.    End for
8.    End for
9.    End for
```

Step count
M+1
M(N+1)
MN(p+1)
MNP
0
0
0
Total = 2MNP+2MN + 2M+ 1

If M == N and M == P then
Total = $2M^3 + 2M^2 + 2M + 1$

Complexity - $O(n^3)$

$$a'a = \begin{bmatrix} 24 & 24 & -6 & -6 & 36 \\ 0 & 30 & 0 & 0 & -30 \\ 30 & -30 & 0 & 30 & -30 \end{bmatrix} \begin{bmatrix} 24 & 0 & 30 \\ 24 & 30 & -30 \\ -6 & 0 & 0 \\ -6 & 0 & 30 \\ -36 & -30 & -30 \end{bmatrix} = \begin{bmatrix} 2520 & 1800 & 900 \\ 1800 & 1800 & 0 \\ 900 & 0 & 3600 \end{bmatrix}$$

Compexity

- Step count method gives the exact number of operations, it becomes cumbersome for large and complex algorithms.
- Loop analysis provides a simpler and more scalable approach.
- Instead of counting every individual statement, we determine how many times a loop executes and multiply it by the cost of the loop body.

Algorithm Efficiency – Single loop

- Linear Loops –

```
i=1
Loop(i<=1000)
    s1;
    s2;
    i=i+1;
End Loop
```

F(n) = n
O(n)

```
i=1
Loop(i<=1000)
    s1;
    s2;
    i=i+2;
End Loop
```

F(n) = n/2
=O(n)

Execution time of an algorithm increases proportionally with the size of the input.

Algorithm Efficiency – Single loop

```
i=1
Loop(i<=64)
    s1;
    s2;
    i=i*2;
End Loop
```

$2^{iterations} = 64$

$Iterations = \log_2 64$

$F(n) = \log_2 n$

$O(\log_2 n)$

```
i=64
Loop(i>=1)
    s1;
    s2;
    i=i/2;
End Loop
```

$2^{iterations} = 64$

$Iterations = \log_2 64$

$F(n) = \log_2 n$

$O(\log_2 n)$

- Logarithmic time complexity means that the execution time increases very slowly as the input size grows.
- Instead of processing every element, the algorithm reduces the problem size by a constant factor (usually half) in each step.

Algorithm efficiency – Nested loop

- Nested (Quadratic) Loops

```
i=1
Loop(i<=100)
    j=1;
Loop(j<=100)
    s1;
    s2;
    j=j+1;
End loop
    i=i+1;
End Loop
```

Inner loop (j) will execute 100 times for each value of i.
Outer loop (i) will execute 100 times

$F(n) = 100 * 100$
$= n * n$
$= n^2$
$= O(n^2)$

If I incremented by 1 and j by 2
j will execute 50 times for each value of i.
i will execute 100 times

$F(n) = 100 * 50$
$= n * n/2$
$= (n^2)/2$
$= O(n^2)$

Quadratic time complexity means that the execution time of an algorithm is proportional to the square of the input size.
As the input size increases, the running time grows much faster than linear time.
Quadratic complexity usually occurs when an algorithm uses two nested loops, where each loop iterates over the input.

Algorithm efficiency – Nested loop

- Nested dependent Linear Loops

```
i=1
Loop(i<=100)
     j=1;
Loop(j<=i)
     s1;
     s2;
     j=j+1;
End loop
     i=i+1;
End Loop
```

Inner loop (j) will execute i times for each value of i.
Outer loop (i) will execute 100 times

$F(n) = 1+2+3.....100$
$= n(n+1)/2$
$= n^2$
$= O(n^2)$

If i incremented by 1 and j by 2.
j will execute 50 times for each value of i.
i will execute 100 times

$F(n) = 100 * 50$
$= n * n/2$
$= (n^2)/2$
$= O(n^2)$

If 'k' linearly incremented loops are nested then complexity is $O(n^k)$ – Polynomial time complexity

Algorithm efficiency – Nested loop

- Nested Linear Logarithmic Loops

```
i=1
Loop(i<=64)
    j=1;
Loop(j<=64)
    s1;
    s2;
    j=j*2;
End loop
    i=i+1;
End Loop
```

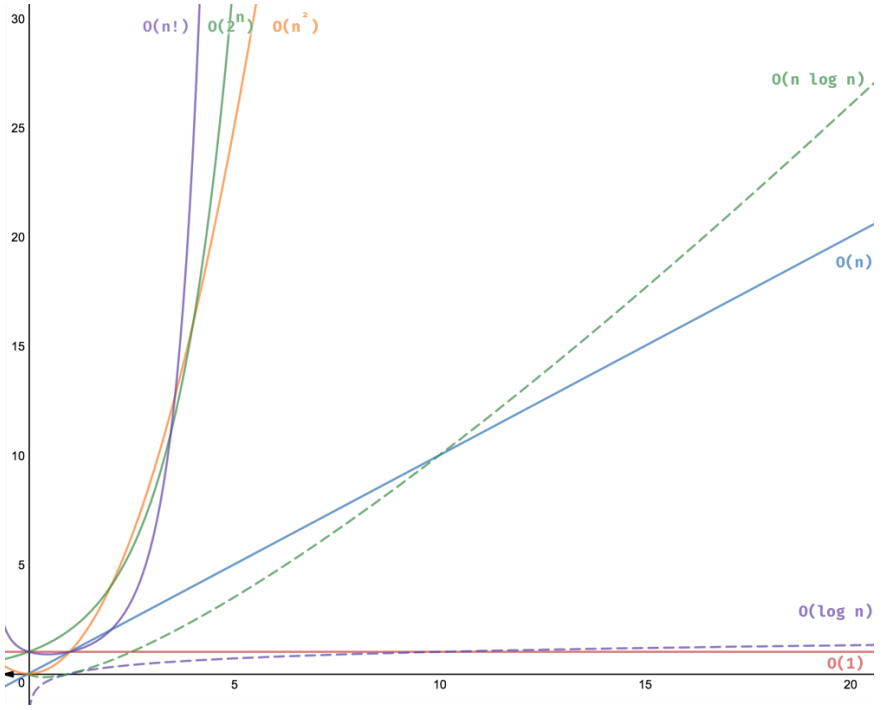
Inner loop (j) will execute $\log_2 64$ times for each value of i.
Outer loop (i) will execute 64 times

$F(n) = 64 * \log_2 64$
$= n * \log_2 n$
$= O(n\log_2 n)$

If j incremented by 1 and i by i*2
j will execute 64 times for each value of i.
i will execute $\log_2 64$ times

$F(n) = \log_2 64 * 64$
$= \log_2 n * n$
$= O(n\log_2 n)$

Graphical representation of algorithm efficiency



Case	Complexity
Best	O(1)
	O(log n)
	O(n)
	O(nlog n)
	$O(n^2)$
	$O(n^3)$
	$O(2^n)$
Worst	O(n!)

Asymptotic Notations

Asymptotic Analysis

- The efficiency of an algorithm depends on the amount of time, storage and other resources required to execute the algorithm. The efficiency is measured with the help of asymptotic notations.
- An algorithm may not have the same performance for different types of inputs. With the increase in the input size, the performance will change.
- The study of change in performance of the algorithm with the change in the order of the input size is defined as asymptotic analysis.

Asymptotic notations

- Asymptotic notations are the mathematical notations used to describe the running time of an algorithm when the input tends towards a particular value or a limiting value.
- Asymptotic notations are mathematical tools to represent the time complexity of algorithms for asymptotic analysis.
- Asymptotic notations are the terminology that will make meaningful statements about time and space complexity.
- Big- O – Upper Bound
- Omega - Ω - Lower bound
- Theta - Θ - average bound

Big oh- Asymptotic Notation

- Big-O notation represents the upper bound of the running time of an algorithm. Thus, it gives the worst-case complexity of an algorithm.
- F(n) – No. of statements executed in function for ‘n’ input.
- F(n) may be defined based on step count or loops in program.
- F(n) = O(g(n)) (read as “F of n is big oh of g of n”) iff there exist positive constant ‘C’ and n_0 such that $F(n) \leq c\ g(n)$ for all n, $n \geq n_0$
- Big-oh expresses upper bound on time complexity.
- It means that if an algorithm is run on same machine but with increasing value of n, the resulting time will always be less than constant time g(n).
- F(n) = 5n+2 g(n) = n c = 6 n_0>=2

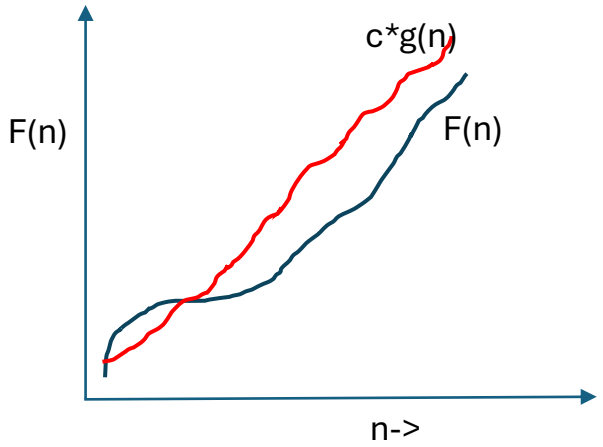
  = O(n)
- Since it gives the worst-case running time of an algorithm, it is widely used to analyze an algorithm as we are always interested in the worst-case scenario.

Big oh- Asymptotic Notation

- F(n) = 5n+2 g(n) = n c = 6 n_0>=2

  = O(n)

- F(n)<= c*g(n) n_0>=2
- 5n+2 <=6n n_0>=2



- 5n+1<=n^2 or n^3 ---- always true but not specifying like this because g(n) should be as nearer to function “F()” of n as one can come up.

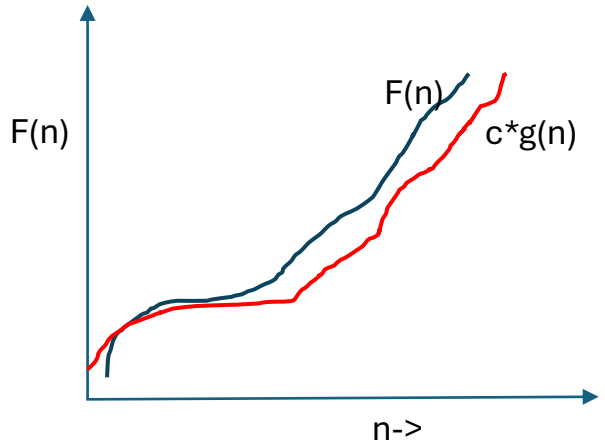
Omega- Asymptotic Notation

- Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best case complexity of an algorithm.
- F(n) – No. of statements executed in function for ‘n’ input.
- F(n) = Ω(g(n)) (read as “F of n is omega of g of n”). iff there exist positive constant ‘C’ and n_0 such that F(n) ≥ c g(n) for all n, n≥ n_0
- Omega expresses lower bound on time complexity.
- It means that if an algorithm is run on same machine but with increasing value of n, the resulting time will always be less than constant time g(n).
- F(n) = 5n+2 g(n) = n c = 5 n_0>=2

  = Ω(n)

Omega- Asymptotic Notation

- F(n) = 5n+2 g(n) = n c = 5 n>=2
  = Ω(n)
- F(n)>= c*g(n) n_0>=2
- 5n+2 >=5n n_0>=2



- 5n+2>=Ω(1) ---- always true but not specifying like this because g(n) should be as near as function “F()” of n as one can come up.

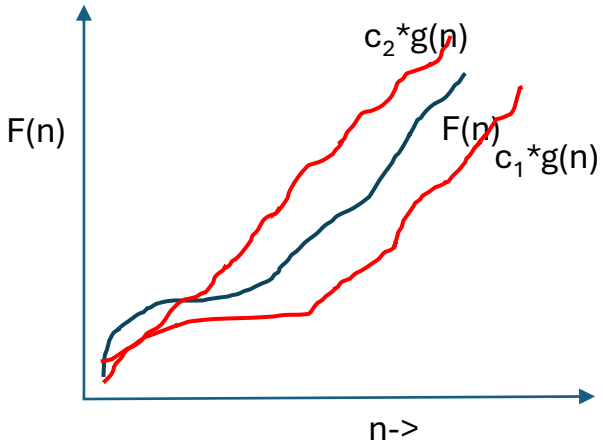
Theta- Asymptotic Notation

- Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm.
- F(n) – No. of statements executed in function for 'n' input.
- F(n) may be defined based on step count or loops in program.
- F(n) = Θ(g(n)) (read as "F of n is big oh of g of n") iff there exist positive constant 'C_1', 'C_2' and n_0 such that $c_1\, g(n) \geq F(n) \leq c_2\, g(n)$ for all n, $n \geq n_0$
- This is more precise than both Big-oh and Omega.
- It means that if an algorithm is run on same machine but with increasing value of n, the resulting time will always be less than constant time g(n).
- F(n) = 5n+2 $\quad g(n)=n,\ c_1\, g(n) = 5n,\ c_2\, g(n) = 6n \quad c_1 = 5,\ c_2 = 6 \quad n_0 >= 2$

  = Θ(n)

Theta Asymptotic Notation

- F(n) = 5n+2 $c_1 g(n) = 5n$, $c_2 g(n) = 6n$ $c_1 = 5$, $c_2 = 6$ $n_0 >= 2$
  = Θ(n)
- $c_1 g(n) <= F(n) <= c_2 g(n)$ $n_0 >= 2$
- $5n <= 5n+2 <= 6n$ $n_0 >= 2$



Abstract Data Type (ADT)

- Definition :- An ADT is Data Type that is organized in such a way that the specification of the objects and the specification of the operations on the objects is separated from the representation of the objects and the implementation of the operations.
- The ADT defines the logical form of the data type and not concerned with implementation.
- It is useful guideline to implementer and useful tool to programmers who wish to use this data type/data structure correctly
- ADT has 2 parts
- 1) Value Definition
  - Definition clause and condition clause
- 2)Operator Definition
  - Header, Precondition, Postcondition

ADT Representation using Class

Class class_name{
Variable list; → value definition in ADT
Public:
List of Functions; → Operator definition in ADT
}

ADT 1D array is
Value Definition : Ele_type a[10] → value definition in ADT

Function Definition :
Void Store(a,i, val);
// stores val in a at ith location
Ele_type Read(a,i); → Operator definition in ADT
// retruns value at ith location

Class Arr {
Int a[10]; → value definition in ADT
Public:
Arr(); → Operator definition in ADT
Arr(int);
Void Store(int,int); //location, value
Int Read(int); //location
}

ADT representation using class for Rational Number

ADT Rational is

Value Definition : A pair of two real numbers Numerator & Denominator

Numerator, denominator are real numbers belong to each object of Rational and denominator not equal to zero

Function Definition :

Rational create(x,y);

// stores x in Numerator, y in denominator ,y<>0

Rational Add(Rational a,Rational b); → Operator definition in ADT

// Performs addition of two Rational number and return

Rational Mul(Rational a,Rational b); → Operator definition in ADT

// Performs Multiplication of two Rational number and return

ADT representation using class for complex number

ADT Complex is

Value Definition :

real, imaginary elements are real numbers belong to each object of complex

Public:

Void create(x,y);

// stores x in real part, y in imaginary part

Complex Add(Complex a,Complex b); → Operator definition in ADT

// Performs addition of two compex number and return

Complex Mul(Complex a,Complex b); → Operator definition in ADT

// Performs multiplication of two compex number and return

•

ADT representation using class for Set

ADT SET is

Value definition

A finite collection with zero or more elements

Function Definition

Set createSet() : creates the empty Set

Void addEle(Set,item) : adds unique items to Set

Set Union(Set,Set): Returns union of 2 sets

Boolean isemptySet(Set): if Set is empty returns TRUE else returns FALSE

void read(); add n elements to set

bool exists(Set, item); if item exists in Set returns true else false

Set Intersection(Set) : Returns Intersection of 2 sets

ADT representation using class for Set

ADT Set is

Value definition : eltype S[10] or *S //A finite collection with zero or more elements

Operator definition :

Set createSet() : creates the empty Set

Void addEle(Set,item) : adds unique items to Set

Set Union(Set,Set): Returns union of 2 sets

Boolean isemptySet(Set): if Set is empty returns TRUE else returns FALSE

void read(); add n elements to set

bool exists(Set, item); if item exists in Set returns true else false

Set Intersection(Set) : Returns Intersection of 2 sets

Class representation for Set

Class Set {

int a[10] → value definition in ADT

Operator

Void Create();

Void addele(int); // add unique elements

Set union(Set s1, Set s2); // perform union of two sets and return resultant to calling function

int length(); → Operator definition in ADT

Set Intersection(Set s1, Set s2);

}

Use of Array

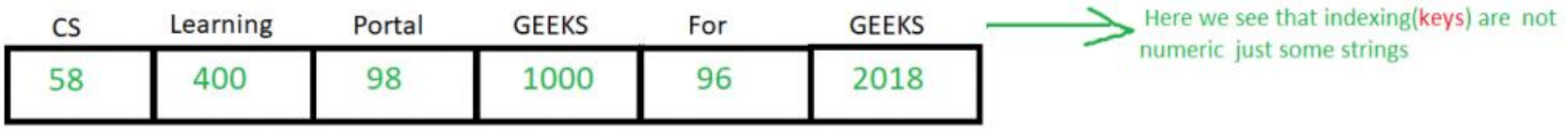
1	Storing Multiple Values	Arrays allow storage of multiple values of the same type using a single variable name Example: int numbers[5] = {10, 20, 30, 40, 50};
2	Random Access	Arrays allow **constant-time (O(1))** access to elements using their index. Example: numbers[2] gives 30.
3	Iteration and Looping	Example: Summing elements, searching, or sorting.
4	Matrix and Multidimensional Structures	2D and 3D arrays are used to represent matrices, tables, and grids
5	Basis for advanced Data Structures	Arrays are the building blocks for many advanced data structures: Strings (character arrays), Stacks (with array implementation), Queues, Hash tables (internal storage), Heaps and priority

Use of Array

6	Efficient Data Management	Example: Storing marks of students
7	Memory Management	Arrays use **contiguous memory**, which helps in faster access and predictable memory usage
8	Real-Life Applications	**Game Development**: Representing maps, character states, or inventory. **Databases**: Internal representation of rows/columns. **Operating Systems**: Managing process tables, memory pages, etc. **Scientific Computing**: Handling large numeric datasets efficiently.

Associative Array

- An associative array is a data structure that stores data as a collection of **key-value pairs**, where each unique key maps to a specific value
- Associative arrays are also called map or dictionaries in C++
- These are special kind of arrays, where indexing can be numeric or any other data type



Example

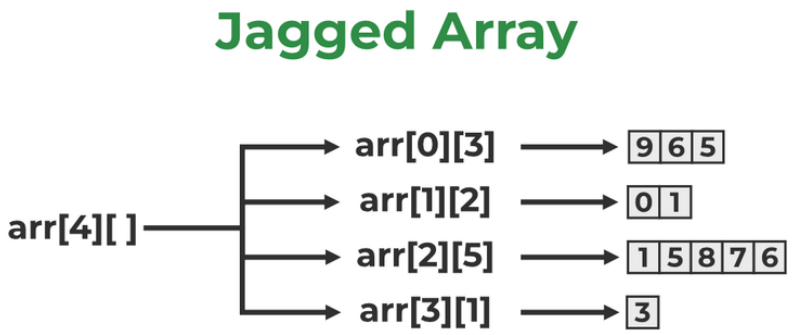
In **C++**, associative arrays are implemented using the **std::map** or **std::unordered_map** containers from the **Standard Template Library**

```
#include <iostream> #include <map>
using namespace std;
int main() {
        map<string, int> studentMarks;
        / /  Inserting elements
        studentMarks[“Ram"] = 85;
        studentMarks[“Geeta"] = 92;
    / /  Accessing elements
        cout << “Ram’s marks: " << studentMarks[“Ram"] << endl;
        / /  Iterating through the map
        for (auto pair : studentMarks) {
            cout << pair.first << " got " << pair.second << " marks."
  << endl;
}
return 0;  }
```

```
#include <iostream> #include <map>
using namespace std;
int main()
{
      map<string, int> marks{ { "Rithvik", 78 },
          { "Suraj", 91 }, { "Jessie", 100 },
          { "Praveen", 99 }, { "Bisa", 84 } };
    map<string, int>::iterator i;
   cout << "The marks of all students are" << endl;
   for (i = marks.begin(); i != marks.end(); i++)
      cout << i->second << " ";
      cout << endl;
   // the marks of the students based on there names.
   cout << "the marks of Computer geek Jessie are"
       << " " << marks["Jessie"] << endl;
   cout << "the marks of geeksforgeeks contributor"
        " Praveen are " << marks["Praveen"] << endl;
}
```

Jagged Array

- A jagged array is an array of arrays such that member arrays can be of different sizes, in 2D array terms for each row we can have a variable number of columns. These types of arrays are also known as Jagged arrays.



Jagged Array

- Jagged Array can be implemented in C++ in two ways:
  - Using a static array of pointers
  - Using dynamic 2D arrays

- **Using a static array of pointers**
  - create 'n' numbers 1D-arrays (row1, row2, row3, ... etc.) where n is no of rows, the size of each row array will be No. of columns i.e, the number of elements in each row array will show no. of columns in that particular row.
  - Create a 1D array of pointers, storing the base address of each row array.
  - Create another 1D array **Sizes[]** storing the size of each row array (This helps while iterating each element)

static array of pointers

create 3 row arrays having different sizes

```
int row1[] = { 1, 2, 3, 4 };

int row2[] = { 5, 6 };

int row3[] = { 7, 8, 9 };

// storing base address of each row array

int* jagged[] = { row1, row2, row3 };

int sizes[] = { 4, 2, 3 };

for (int i = 0; i < 3; i++) {

int* ptr = jagged[i];

for (int j = 0; j < sizes[i]; j++) {

cout << *(ptr + j) << " ";

        }}
```

Jagged Array

- **Using Dynamic 2D arrays**
  - Create row, col variables storing no. of rows and no. of columns.
  - Create a Dynamic row array arr (array of pointers) that can store a "row" number of addresses.
  - Store size in another 1D array Sizes[] (size of no. or Rows) to store no. of columns for each row element in the row array. (This helps while iterating each column for each row).
  - Create a Dynamic Column array for each row element with size: sizes[i]
  - Now each row element in the Row array has the Base address of each Column array.

Dynamic 2D array

```
 int row, col;

 row = 3;

 // Create Row Array- dynamic array of pointers

    int** arr = new int*[row]

    int *sizes= new int[row];

for (int i = 0; i < row; i++) {

        cin>>col;        // take col is taken as input

        sizes[i]=col;

       *(arr + i) = new int[sizes[i]];

}
```

```
for (int i = 0; i < row; i++)

{

       for (int j = 0; j < sizes[i]; j++)

      {

          cin>>arr[i][j];

       }

 }
```

Searching Methods

Searching Methods

- Searching Algorithm:- Searching Algorithm is an algorithm that retrieves information stored within some data structure or calculated in the search space of a problem domain.
- Unordered (unsorted) data
  - Linear Search
  - Sentinel Search
- Ordered (Sorted) data
  - Binary Search
  - Fibonacci Search

Linear Search

- Linear search, also called as sequential search, is a very simple method used for searching an array for a particular value
- It works by comparing the value to be searched with every element of the array one by one in a sequence until a match is found
- Linear search is mostly used to search an unordered list of elements

Algorithm

- Step 1: Start from the leftmost element of the array and, one by one, compare x with each element of the array.
- Step 2: If x matches with an element, return the index.
- Step 3: If x doesn't match any of the elements, return -1.

Linear Search

- Assume following Array

0	1	2	3	4	5	6	7	8	← Index
10	76	45	23	88	12	79	26	-43	← Value

- Search 88 ,index = 0

0	1	2	3	4	5	6	7	8
10	76	45	23	88	12	79	26	-43

- Return index 4
- Average case – searching element other than 1st or last element
- Best case

0	1	2	3	4	5	6	7	8
10	76	45	23	88	12	79	26	-43

- Worst case

0	1	2	3	4	5	6	7	8
10	76	45	23	88	12	79	26	-43

Pseudocode – Linear Search

int search(ele list[], int count, ele key)

Precondition: list is collection of elements of same datatype, count represents number of elements in list, key is element to be searched

Postcodintion: Returns position of key in the list otherwise -1

1. found := false
2. position := –1, index := 0
3. While (index < count)
   3.1 If (key == list[index]) Then
   found := true
   position := index
   break;
   End If
   3.2 Index:=index + 1
4. End While
5. Return position

1. 1
2. 1
3. N

3.1 N-1

3.2 N-1

5. 1

Total – 3N+1

Complexity

- Time Complexity
  - Best Case – O(1)
  - Worst Case – O(n)
  - Average Case – O(n)

- Space Complexity: O(1)

Advantages and Limitations

- Advantages of Linear Search
  - Simple and easy to implement.
  - Can be applied to unsorted and unstructured data.
  - Requires no pre-processing or ordering of data.

- Limitations
  - Inefficient for large datasets.
  - Time complexity increases linearly with the size of the dataset.

Sentinel Search

- The Sentinel Linear Search is an optimized version of linear search that reduces the number of comparisons.
- It does this by placing a special marker, called a sentinel, at the end of the array, ensuring the search will always find the target (or its placeholder) without unnecessary checks.
- Key Idea: Instead of checking for both element match and end of list conditions separately, the sentinel guarantees termination, thus eliminating one comparison per iteration.

Sentinel Search Cont...

- The Sentinel Linear Search follows these steps:
  - Append a sentinel value (the target element itself) at the end of the array.
  - Start searching sequentially from the beginning.
  - If the target is found before reaching the sentinel, return the index.
  - If the sentinel is encountered, determine whether the search was successful or unsuccessful.
  - Restore the original array state if modifications were made for sentinel placement.

Sentinel Search

- Assume following Array, x = number to be searched= 88

0	1	2	3	4	5	6	7	8	← Index
10	76	45	23	88	12	79	26	-43	← Value

- Store last = -43 and replace a[n-1] with x

0	1	2	3	4	5	6	7	8
10	76	45	23	88	12	79	26	88

- index = 4, replace a[n-1] = last

0	1	2	3	4	5	6	7	8
10	76	45	23	88	12	79	26	-43

- Return index 4 as 4 is less than n-1
- Average case – searching element other than 1st or last element
- Best Case – Element to be searched is First element

Sentinel Search

- Assume following Array, x = number to be searched= -433

0	1	2	3	4	5	6	7	8
10	76	45	23	88	12	79	26	-433

← Index

← Value

- Store last = -43 and replace a[n-1] with x

0	1	2	3	4	5	6	7	8
10	76	45	23	88	12	79	26	-433

- index = 8, replace a[n-1] = last

0	1	2	3	4	5	6	7	8
10	76	45	23	88	12	79	26	-43

- index is not less than n-1 and x is also not equal to a[n-1]
- Worst case – N + 2 comparison are done
- Worst case of linear search is 2N

Pseudocode

- Procedure Sentinel_Search(ele arr[], int n, ele x)

Precondition: ‘arr’ is collection of elements of same datatype, ‘n’ represents number of elements in ‘arr’, ‘x’ is element to be searched

Postcodintion: Display index of x in the arr otherwise “Not found”

```
1.  int last := arr[n - 1];    // Last element of the array
2.  arr[n - 1] := x;      // Element to be searched is placed at last index
3.  int i = 0;
4.  while (arr[i] != x)
5.          i++;
6.  arr[n - 1] := last;      // Put the last element back
7.  if ((i < n - 1) || (x == arr[n - 1]))
8.          cout << x << " is present at index " << i;
9.  Else
10.         cout << "Not found“
11. End Sentinel_search
```

1. 1
2. 1
3. 1
4. N
5. N-1
6. 1
7. 1
8. or 10 1

Total – 2N+5

Complexity

- Time Complexity
  - Best Case – O(1)
  - Worst Case – O(n)
  - Average Case – O(n)

- Compared to basic linear search, sentinel search is slightly faster in practice because it avoids checking the array bounds on every iteration

Advantages and Limitations

- Advantages
  - Faster Execution: Eliminates unnecessary boundary checks, slightly improving speed.
  - Simple Implementation: Minimal modifications make it easy to integrate into existing systems.
  - Efficient for Small Datasets: Works well for small lists where advanced algorithms are unnecessary.
  - Works on Unsorted Data: Like traditional linear search, it does not require a sorted list.
- Limitations
  - Still O(n) Complexity: Does not significantly improve efficiency for large datasets.
  - Modifies Input Temporarily: Although restored, modifying input data can be undesirable in some applications.
  - Not Ideal for Large Searches: More efficient algorithms like binary search or hash-based search should be preferred when applicable.

Binary Search

- Search a sorted array by repeatedly dividing the search interval in half.
- It begins with an interval covering the whole array.
- If the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half.
- If the value of the search key is greater than the item in the middle of the interval, narrow it to the upper half.
- Repeatedly check until the value is found or the interval is empty.
- The algorithm ignores half of the elements just after one comparison.
- Upper bound of Loop is decreasing by n/2 after each comparison.
- Time complexity is $O(Log_2 n)$.

Binary Search

- Assume following Sorted Array start=0,end = 8

0	1	2	3	4	5	6	7	8
10	20	30	40	50	60	70	80	90

← Index

← Value

- Search x = 80 , start = 0, end = 8, mid = 4

0	1	2	3	4	5	6	7	8
10	20	30	40	50	60	70	80	90

- As x is >A[mid]
- Start = mid+1 = 5 end = 8
- Mid = ceil(start+end)/2) = 6

0	1	2	3	4	5	6	7	8
10	20	30	40	50	60	70	80	90

- As x is >A[mid]
- Start = mid+1 = 6 end = 8
- Mid = ceil(start+end)/2) = 7
- X ==A[mid] return mid

Binary Search

- Assume following Sorted Array start=0,end = 8
- Search x = 5 , start = 0, end = 8, mid = 4

0	1	2	3	4	5	6	7	8
10	20	30	40	50	60	70	80	90

← Index

← Value

0	1	2	3	4	5	6	7	8
10	20	30	40	50	60	70	80	90

- As x is <A[mid]
- Start = 0 end = mid -1 = 3
- Mid = ceil((start+end)/2) = 1

0	1	2	3	4	5	6	7	8
10	20	30	40	50	60	70	80	90

- As x is <A[mid]
- Start = 0 end = mid -1= 0
- Mid = 0
- X !=A[mid] and start==end
- Return number not found

Pseudocode Iterative

```
Procedure binarySearch(ele arr[50], int s, int e, ele x)
{
Precondition :- arr should be sorted array
Postcondition :- will return position of x if present otherwise -1
Int m;
while (s<= e)
{
m =floor((s+e) / 2);    //find mid element

if (arr[m] == x)      // Check if x is present
                          at mid
   return m;

if (arr[m] < x)    // If x greater,
                       ignore left half
        s := m + 1;
Else
        e := m - 1;    // If x is smaller,
                           ignore right half
}
 return -1;    // if we reach here,
               then element was not
                present

}
```

Pseudocode - Recursive

```
Procedure binarySearch(int arr[], int s, int e, int x) {
Precondition :- arr should be sorted array
Postcondition :- will return position of x if present otherwise -1
if(s<=e)
{
int m :=floor((s+e) / 2);
if (arr[m] == x)                    // Check if x is present at mid
        return m;
if (arr[m] < x)                     // If x greater, ignore left half
        s := m + 1;
        return(binarySearch(arr,s,e,x));
else
        e := m - 1;                 // If x is smaller, ignore right half
        return(binarySearch(arr,s,e,x));
}
else
         return -1;                 // if we reach here, then element was not present
}
```

Complexity

- Time complexity
- At each iteration the size of array decrease by half.
- So While loop is iterating at most $\log_2 n$
  - Best case – O(1)
  - Average, worst case - $O(\log_2 n)$

- Space complexity
  - Best case – O(1)
  - Average, worst case - $O(\log_2 n)$

Advantages and Limitations

- Advantages
  - More efficient for large datasets compared to linear search.
  - Reduces the number of comparisons significantly by halving the data set at each step.

- Limitations
  - Requires a sorted dataset.
  - Additional overhead for sorting the data if it is not sorted.

Fibonacci Search

- Fibonacci Search is a comparison-based technique that uses Fibonacci numbers to search for an element in a sorted array
- It is very similar to binary search
- Works on sorted arrays
- A Divide and Conquer Algorithm
- Time complexity Log_2n
- Binary search divides the given array at mid but Fibonacci search divides in unequal parts
- Binary Search uses the division operator to divide the range. Fibonacci Search doesn't use / but uses + and -. The division operator may be costly on some CPUs
- Fibonacci Search examines relatively closer elements in subsequent steps. So, when the input array is big that cannot fit in the CPU cache or even in RAM, Fibonacci Search can be useful

How Fibonacci Search works?

1. Find the immediate Fibonacci number F_m that is greater than or equal to the size of the input array. Also hold the two preceding numbers of the selected Fibonacci number (F_m) F_{m-1}, F_{m-2} from the Fibonacci Series.

2. Initialize the offset value as -1, as we are considering the entire array as the searching range in the beginning.

3. Until Fm-2 is greater than 0, we perform the following steps –
   i. Compare the key element to be found with the element at index **[min(offset+F_{m-2},n-1)]**. If a match is found, return the index.
   ii. If the key element is found to be lesser value than this element, the Fibonacci numbers are also updated with $F_m = F_{m-2}$.
   iii. But if the key element is greater than the element at this index, the Fibonacci numbers are updated as $F_m = F_{m-1}$. The *offset* value is set to the index of this element.

Fibonacci Search Pseudocode

Procedure Fibonacci_Search(A, x, N)

Precondition :- arr should be sorted array

Postcondition :- will return position of x if present otherwise -1

```
offset = -1
while fm > 1 do:
    i := minimum of (offset + fm2, n -1)
    if (A[i] < x) then:
      Fm := Fm1
      Fm1 := Fm2
      Fm2 := Fm - Fm1
      offset = i
    end
else if (A[i] > x) then:
      Fm = Fm2
      Fm1 = Fm1 - Fm2
      Fm2 = Fm - Fm1
    end
else
      return i;
end
if (!Fm1 and Array[offset + 1] == x) then:
    return offset + 1
  end
    return -1
End
```

Complexity

- Time complexity
  - At each iteration the size of array decrease by half.
  - So While loop is iterating at most $\log_2 n$
  - Best case – O(1)
  - Average, worst case - $O(\log_2 n)$
- Space complexity
  - Best case – O(1)
  - Average, worst case - $O(\log_2 n)$

Hashing

Introduction

- Hashing
  - A technique for search, insert, and delete operations that has very good expected performance.

- Applications:
  - Symbol table of a compiler.
  - Memory-management tables in operating systems.
  - Large-scale distributed systems.

Search Techniques

Search tree methods

- Identifier comparisons

Hashing methods

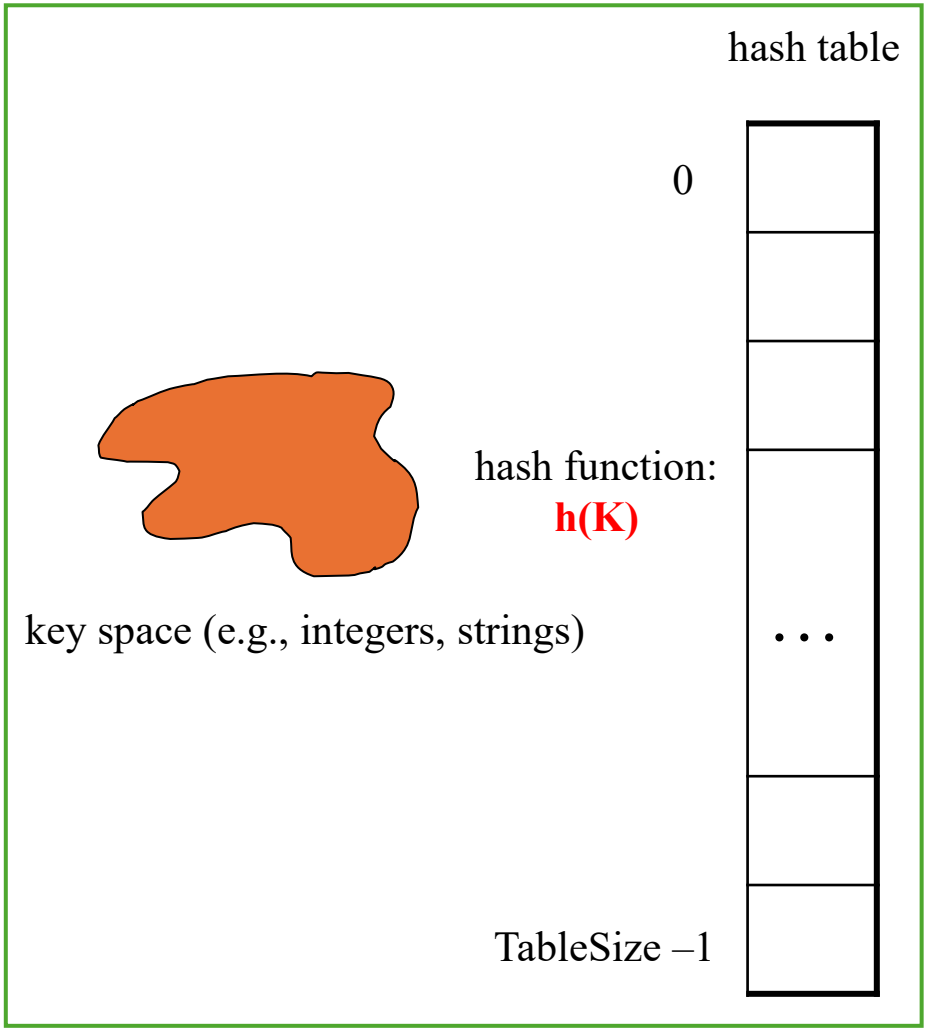
- Relies on a formula called the hash function.

Types of hashing

- Static hashing
- Dynamic hashing

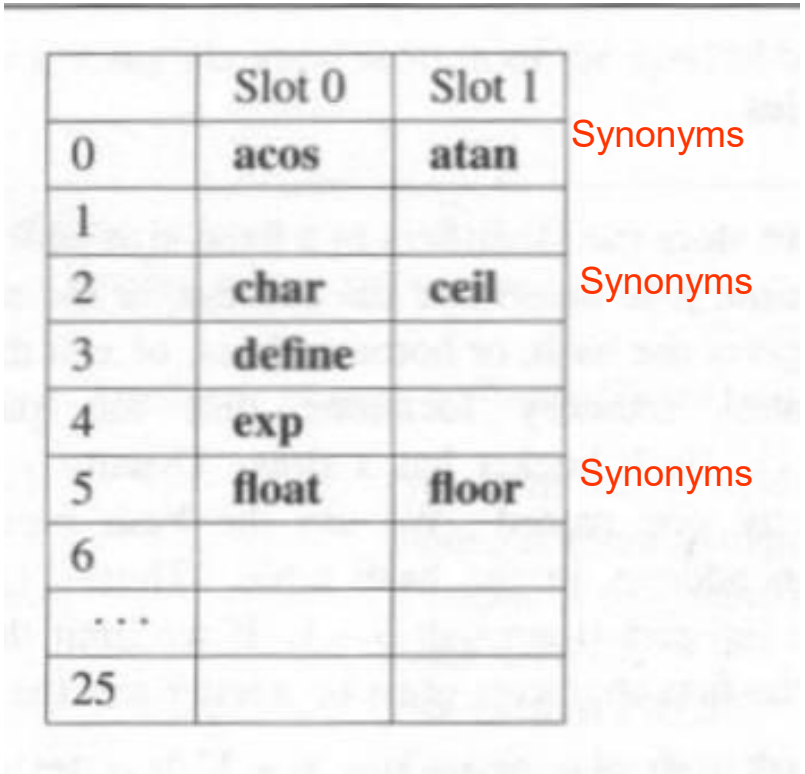
Static Hashing

- In static hashing, the identifiers are stored in a fixed size table called a *hash table*
- Arithmetic function, f
  - To determine the address of an identifier, x, in the table
  - h(x) gives the hash, or home address, of x in the table
- Hash table, ht
  - Stored in sequential memory locations that are partitioned into b buckets, ht[0], ..., ht[b-1].
  - Each bucket has s slots



Synonym

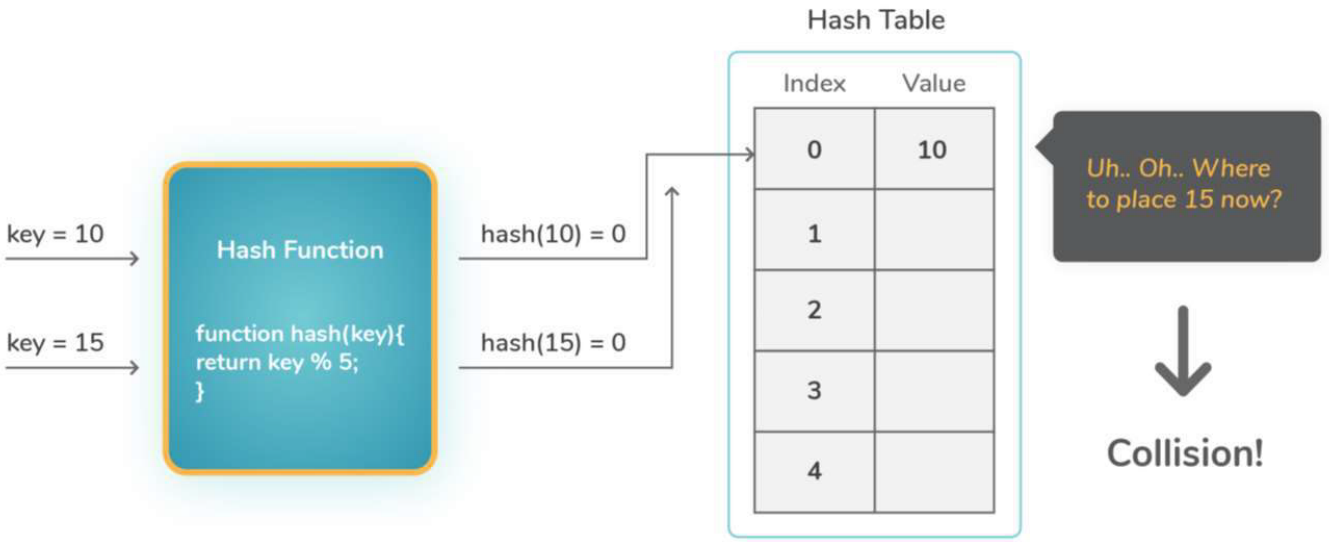
- Two identifiers, i_1 and i_2 are synonyms with respect to h if $h(I_1) = h(I_2)$
  - We enter distinct synonyms into the same bucket as long as the bucket has slots available
- An overflow occurs when a new identifier is mapped by h(x) into a full bucket



	Slot 0	Slot 1	
0	acos	atan	Synonyms
1			
2	char	ceil	Synonyms
3	define		
4	exp		
5	float	floor	Synonyms
6			
...			
25			

Collision

- A collision occurs when two non-identical identifiers are hashed into the same bucket.
- When the bucket size is 1, collisions and overflows occur simultaneously.



- Time required to enter or search for identifiers using hashing depends only on the time required to compute the hash function and time to search one bucket.

- Sequential search is carried out to search bucket for an identifier

Hashing function

1.Division

- The identifier x is divided by size of the table & remainder is used as hash address of x
- h(x)= x mod m

- e.g m=10
- key(13) = 13 mod 10=3

- Prime number array size produces fewer collision

Hashing function

2.Multiplication

n(key) = floor(m * frac(c*key))

where

real no c between 0 &1 is selected

floor (x)= integer part of real no x

frac(x) = fractional part [x-floor[x]]

m=size of the hash table

e,g key 68 c=0.6180

n(68) = floor (256 * frac (0.6180*68))

= floor(256*frac(42.024))

=floor(256*0.024)

=6

Hashing function

3.Mid square method

Key is squared and then mid part of the result is taken as the index.

The number of bits to be used to obtain bucket address depends on table size.

If r bits are used, the range of values is 0 trough 2^r-1.

Steps:

- Take the key K.
- Compute its square (K^2).
- Extract the middle digits (or bits) of the squared value.
- Use these middle digits as the hash address

Example

Suppose the hash table size is 100 (indices 0–99).

Key = 123

Step 1: Square the key $123^2 = 15129$

Step 2: Extract the middle two digits

Hash Address = 12

Suppose the hash table size is 1000 (indices 0–999).

Key = 3102

Step 1: Square the key $3102^2 = 9616201$

Step 2: Extract the middle three digits

Hash Address = 162

Hashing function

4.Folding

Partition the identifier x into several parts

All parts except for the last one have the same length

Add the parts together to obtain the hash address

E.g k = 12320324111220

Two possibilities

a) Shift folding

x1=123, x2=203, x3=241, x4=112, x5=20, address=699

b) Folding at the boundaries

x1=123, x2=302, x3=241, x4=211, x5=20, address=897

Imp. Points to remember when choosing hash function

1. Computing efficiency: The hash function should compute the hash value quickly and efficiently even for large key values.
2. Uniform distribution: The hash function should be able to distribute the keys evenly in the hash table.
3. Minimal Collisions: The hash function should minimize the key collisions in the hash table

Collision Resolution Strategies

- Two ways:

1. Open Addressing
2. Chaining

Open Addressing

- In this scheme, during insertion of new item into hash table, if the hash function indicates a location in the hash table is already occupied, we probe for some other empty location in which to place the item.

- The sequence of locations that we examine is called the Probe sequence.

- Three different open-addressing schemes:
  - Linear Probing
  - Quadratic Probing
  - Double Hashing

Linear Probing or Linear Open Addressing

Insert Algorithm:

```
i <- hash(key)
loop
  if HT[i] is empty then
     HT[i] <- key
     break;
  else
     i <- (i + 1) mod size_of_HT
end loop
```

Cont’d

- When linear open addressing is a used to handle overflows, a hash table search for identifier 'x' proceeds as follows:

1. compute h(x)
2. examine identifiers at positions ht[h(x)], ht[h(x)=1], ...ht[h(x)+j] in this order, until one of the following happens,
   a) ht[h(x)+j] = x — x is found
   b) ht[h(x)+j] is null — x is not in the table
   c) we return to the starting position h(x); the table is full and 'x' is not in the table

Linear Probing without Replacement

Table size=10
h(x)= x mod 10

h(20) = 20 mod 10=0

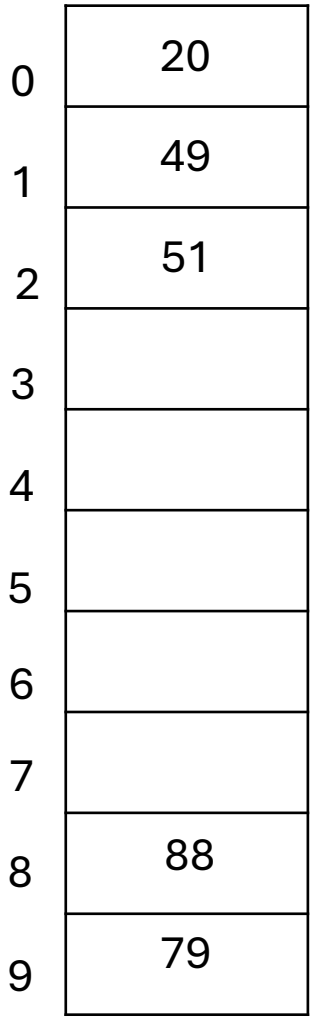
h(88) = 88 mod 10=8

h(79) = 79 mod 10=9

h(49) = 49 mod 10=9

Since bucket 9 is already occupied, so collision occurs. To handle the collision, linear probing technique keeps probing linearly until an empty bucket is found. The first empty bucket is bucket 2.

h(51) = 51 mod 10=1



Linear Probing with Replacement

Table size=10
h(x)= x mod 10

h(20) = 20 mod 10=0

h(88) = 88 mod 10=8

h(79) = 79 mod 10=9

h(49) = 49 mod 10=9

Since bucket 9 is already occupied, collision occurs. Check whether correct identifier is present in bucket 9 with hash address 9. If yes, linear probing keeps probing linearly until an empty bucket is found. The first empty bucket is bucket 2.

h(51) = 51 mod 10=1

Identifier 49 is present in bucket 2 with hash address 9. Since the hash and bucket address is not same, find the next empty location for 49 and insert 51 in bucket 1.

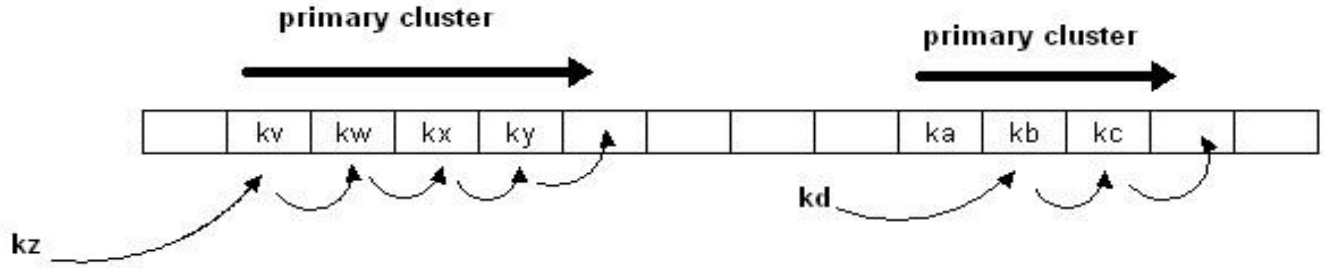
0	20
1	51
2	49
3	
4	
5	
6	
7	
8	88
9	79

Advantages

- Linear probing requires very less memory.
- It is less complex and is simpler to implement.

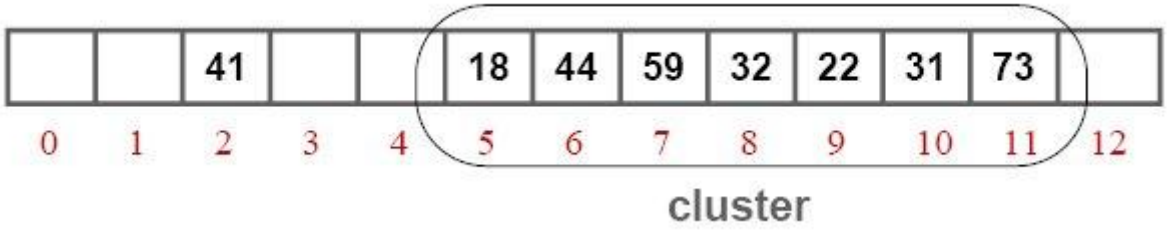
Disadvantage of Linear Probing

- Linear probing is subject to a primary clustering phenomenon.
- Elements tend to cluster around table locations that they originally hash to.
- Primary clusters can combine to form larger clusters. This leads to long probe sequences and hence deterioration in hash table efficiency.



Example of a primary cluster: Insert keys: **18, 41, 22, 44, 59, 32, 31, 73**, in this order, in an originally empty hash table of size **13**, using the hash function **h(key) = key % 13** and **c(i) = i**:

h(18) = 5
h(41) = 2
h(22) = 9
h(44) = 5+1
h(59) = 7
h(32) = 6+1+1
h(31) = 5+1+1+1+1+1
h(73) = 8+1+1+1



```
 int hash(int key)
{
   return key % SIZE;
}

void LPWithoutReplacement(int key)
{
   int home, current;

   home = hash(key);

   // Check if home location is empty
   if (hashTable[home] == EMPTY)
   {
      hashTable[home] = key;
      return;
}
```

```
  // Search for next empty location
    current = home;
     do
    {
       current = (current + 1) % SIZE;
    }
    while (!hashTable[current] != EMPTY
&& current != home);

    // Check for overflow
    if (current == home)
    {
       printf("Hash Table Overflow\n");
    }
    else
    {
       hashTable[current] = key;
    }
}
```

```
void LPWithReplacement(int key)
{
   int home, current, existing;

   home = hash(key);

   // If home location is empty
   if (hashTable[home] == EMPTY)
   {
      hashTable[home] = key;
      return;
   }
    existing = hashTable[home];
    // Existing key belongs to its home location
   if (hash(existing) == home)
   {
      current = home;
       do
      {
         current = (current + 1) % SIZE;
      } while (hashTable[current] != EMPTY && current != home);
      if (current == home)
      {
         printf("Hash Table Overflow\n");
      }
      else
      {
         hashTable[current] = key;
      }
   }
  else
   {
     hashTable[home] = key;

      current = hash(existing);

      while (hashTable[current] != EMPTY)
      {
         current = (current + 1) % SIZE;
      }

      hashTable[current] = existing;
   }
}
```

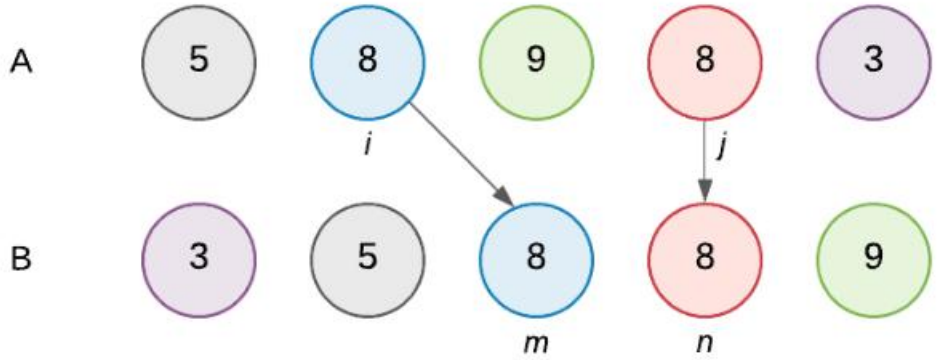
Sorting Algorithms

Sorting Methods

- Arrange data in ascending or descending order.
- Types
  - Internal Sort Methods
    - The data that we are sorting is stored in main memory
  - External Sort Methods
    - The part of data that we are sorting is stored in auxiliary memory
- Stability
- In-place sorting

Stable Sorting

- A sorting technique is called a stable sort if for all records i and j such that k[i] equals k[j], if r[i] precedes r[j] in the original file, r[i] precedes r[j] in the sorted file.
- Stable sort keeps records with the same key in the same relative order that they were in before the sort.



- Stable sorting maintains the order of the two equal balls numbered 8

In-place Sorting and Not-in-place Sorting

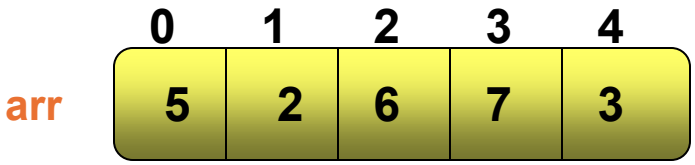
- Sorting algorithms may require some extra space for comparison and temporary storage of few data elements.
- The algorithms do not require any extra space and sorting is said to happen in-place, or for example, within the array itself. This is called **in-place sorting**.
- In some sorting algorithms, the program requires space which is more than or equal to the elements being sorted.
- Sorting which uses equal or more space is called **not-in-place sorting**.

Bubble sort

- Bubble sort is a simple sorting algorithm.
- This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared
- The elements are swapped if they are not in order.
- If n elements are then n-1 iterations required to sort all elements
- n-1 comparisons occurs for each iterations

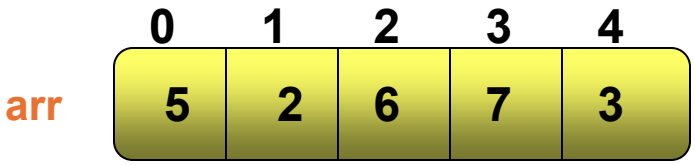
Implementing Bubble Sort Algorithm

- To understand the implementation of bubble sort algorithm, consider an unsorted list of numbers stored in an array.



Implementing Bubble Sort Algorithm (Contd.)

- Let us sort this unsorted list.

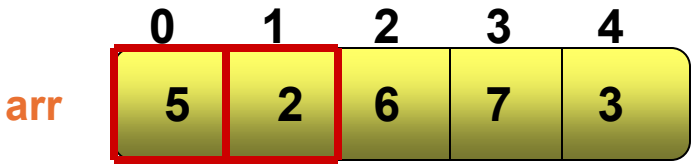


Implementing Bubble Sort Algorithm (Contd.)

Pass 1

n = 5

- Compare the element stored at index 0 with the element stored at index 1.

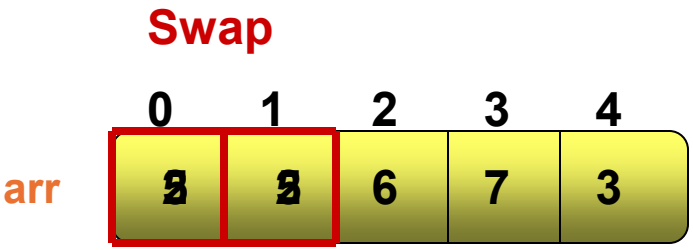


Implementing Bubble Sort Algorithm (Contd.)

Pass 1

n = 5

- Swap the values if they are not in the correct order.



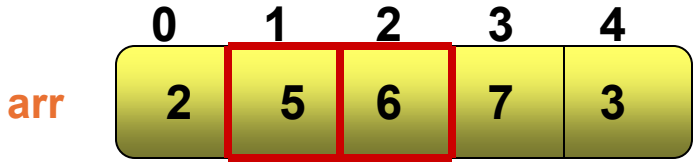
Implementing Bubble Sort Algorithm (Contd.)

Pass 1

n = 5

- Compare the element stored at index 1 with the element stored at index 2 and swap the values if the value at index 1 is greater than the value at index 2.

No Change

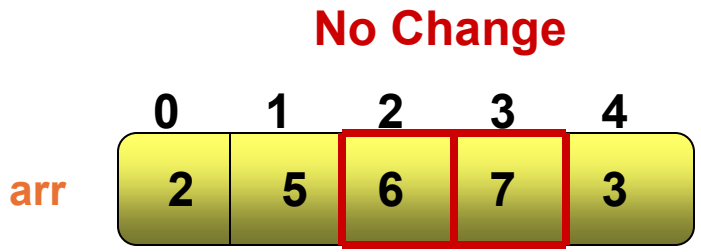


Implementing Bubble Sort Algorithm (Contd.)

Pass 1

n = 5

- Compare the element stored at index 2 with the element stored at index 3 and swap the values if the value at index 2 is greater than the value at index 3.

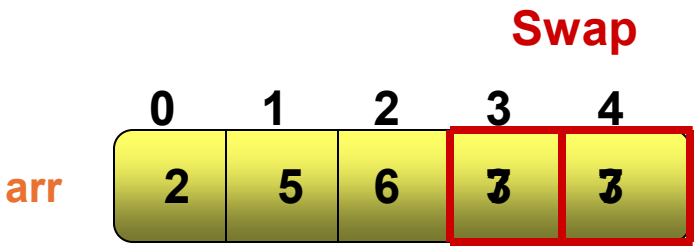


Implementing Bubble Sort Algorithm (Contd.)

Pass 1

n = 5

- Compare the element stored at index 3 with the element stored at index 4 and swap the values if the value at index 3 is greater than the value at index 4.

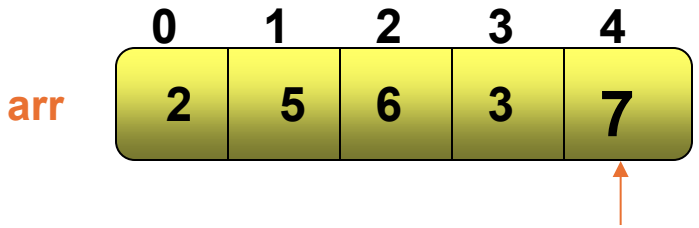


Implementing Bubble Sort Algorithm (Contd.)

Pass 1

n = 5

- Compare the element stored at index 3 with the element stored at index 4 and swap the values if the value at index 3 is greater than the value at index 4.



Largest element is placed at its correct position after Pass 1

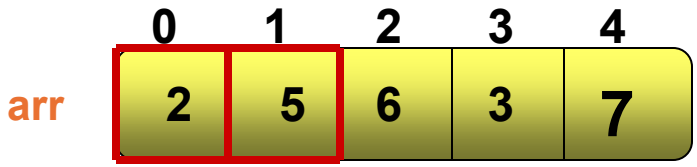
Implementing Bubble Sort Algorithm (Contd.)

Pass 2

n = 5

- Compare the element stored at index 0 with the element stored at index 1 and swap the values if the value at index 0 is greater than the value at index 1.

No Change



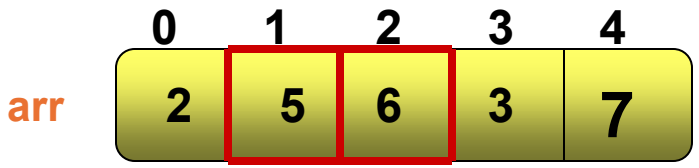
Implementing Bubble Sort Algorithm (Contd.)

Pass 2

n = 5

- Compare the element stored at index 1 with the element stored at index 2 and swap the values if the value at index 1 is greater than the value at index 2.

No Change

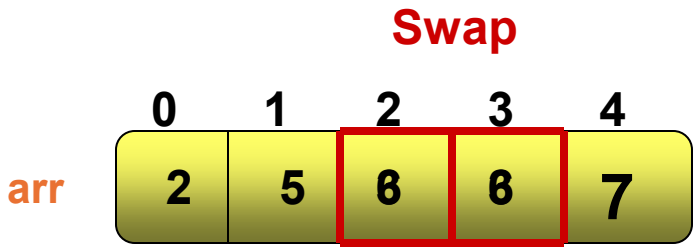


Implementing Bubble Sort Algorithm (Contd.)

Pass 2

n = 5

- Compare the element stored at index 2 with the element stored at index 3 and swap the values if the value at index 2 is greater than the value at index 3.

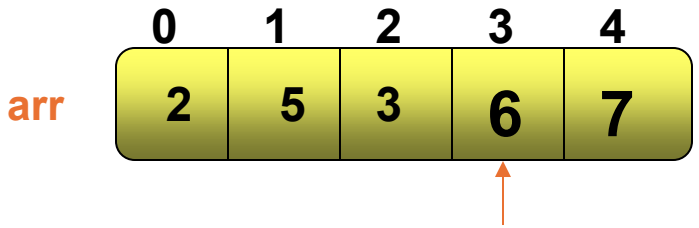


Implementing Bubble Sort Algorithm (Contd.)

Pass 2

n = 5

- Compare the element stored at index 2 with the element stored at index 3 and swap the values if the value at index 2 is greater than the value at index 3.



Second largest element is placed at its correct position after Pass 2

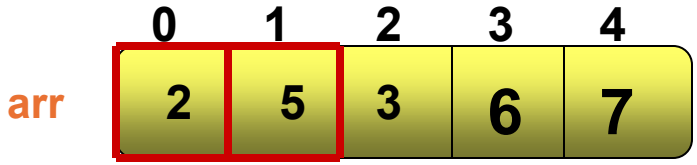
Implementing Bubble Sort Algorithm (Contd.)

Pass 3

n = 5

- Compare the element stored at index 0 with the element stored at index 1 and swap the values if the value at index 0 is greater than the value at index 1.

No Change

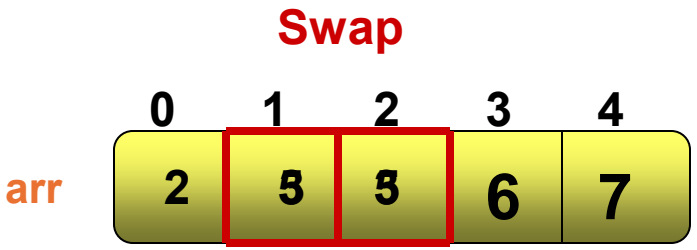


Implementing Bubble Sort Algorithm (Contd.)

Pass 3

n = 5

- Compare the element stored at index 1 with the element stored at index 2 and swap the values if the value at index 1 is greater than the value at index 2.

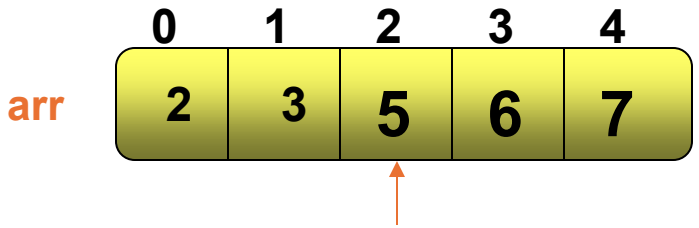


Implementing Bubble Sort Algorithm (Contd.)

Pass 3

n = 5

- Compare the element stored at index 2 with the element stored at index 3 and swap the values if the value at index 2 is greater than the value at index 3.



Third largest element is placed at its correct position after Pass 3

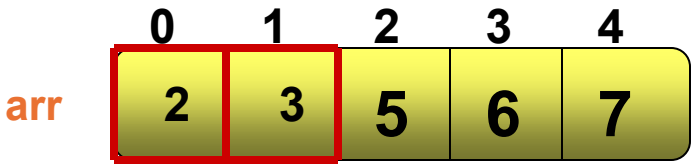
Implementing Bubble Sort Algorithm (Contd.)

Pass 4

n = 5

- Compare the element stored at index 0 with the element stored at index 1 and swap the values if the value at index 0 is greater than the value at index 1.

No Change

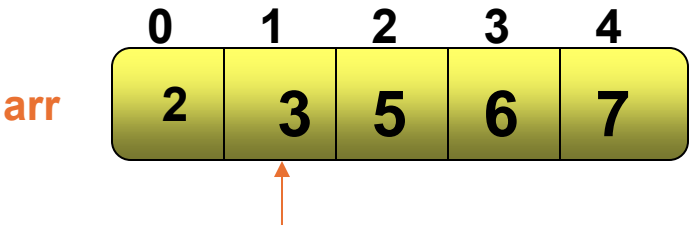


Implementing Bubble Sort Algorithm (Contd.)

Pass 4

n = 5

- Compare the element stored at index 0 with the element stored at index 1 and swap the values if the value at index 0 is greater than the value at index 1.



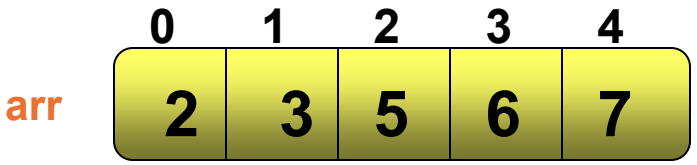
Fourth largest element is placed at its correct position after Pass 4

Implementing Bubble Sort Algorithm (Contd.)

Pass 4

n = 5

- At the end of Pass 4, the elements are sorted.



Bubble Sort

```
procedure bubbleSort( int list[], int n )

  1. for i = 0 to n-1
  2.  for j = 0 to n-1
          2.1 if list[j] > list[j+1] then          /* compare the adjacent elements */
              2.1.1 swap( list[j], list[j+1] )          /* swap them */
         2.2end if
  3. end for
  4. end for
end procedure
```

Iterations – n-1
Comparisons – n-1 for each iteration
Total comparisons – (n-1)(n-1)= n^2-2n+1

Bubble Sort

```
procedure bubbleSort( int list[], int n )

  1. for i = 0 to n-1
  2.    for j = 0 to n-i-1
              2.1 if list[j] > list[j+1] then          /* compare the adjacent elements */ n-1
                    2.1.1 swap( list[j], list[j+1] ) /* swap them */
            2.2 end if
   3. end for

  4. end for
end procedure
```

Iterations – n-1
Comparisons – n-1 for 1st iteration, n-2 for 2nd iteration,.....1comparison for (n-1) comparison
Total comparisons – 1+2+3+.....+(n-1)= n*(n+1)/2

Time Complexity - $O(n^2)$

Bubble Sort

```
procedure bubbleSort( int list[], int n )
  1. for i = 0 to n-1
  2. for j = 0 to n-i-1
                3. if list[j] > list[j+1] then          /* compare the adjacent elements */
                     3.1 swap( list[j], list[j+1] )          /* swap them */
                     3.2 swapped = true
           4. end if
   5. end for
// if no number was swapped that means array is sorted now, break the loop.
    6. if(not swapped) then
        6. 1   break
     7. end if
  8. end for
end procedure
```

Iterations – n-1

Comparisons – n-1 for 1st iteration, n-2 for 2nd iteration,.....1comparison for (n-1) comparison

Total comparisons – 1+2+3+.....+(n-1)= n*(n+1)/2

Time Complexity - $O(n^2)$ – average, worst

- O(n) - Best

Bubble sort

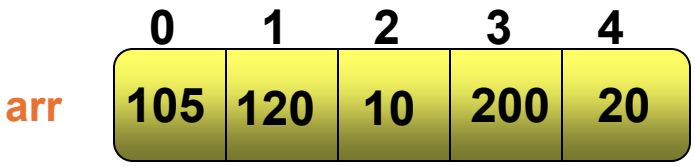
- Bubble sort is internal sort method
- It is stable algorithm
- It is in-place sorting method
- Time Complexity (Best): $O(n^2)$
- Time Complexity (Average): $O(n^2)$
- Time Complexity (Worst): $O(n^2)$
- Space Complexity: O(1) – in-place sort

Selection Sort

- Selection sort is a simple sorting algorithm.
- This sorting algorithm is an in-place comparison-based algorithm
- It is divided into two parts, the sorted part at the left end and the unsorted part at the right end.
- Initially, the sorted part is empty and the unsorted part is the entire list.
- The smallest element is selected from the unsorted array and swapped with the leftmost element, and that element becomes a part of the sorted array.
- This process continues moving unsorted array boundary by one element to the right.

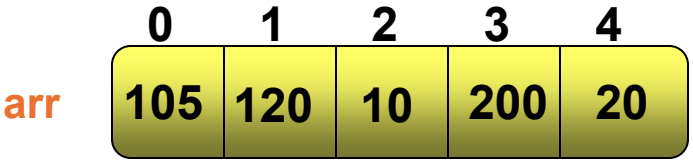
Implementing Selection Sort Algorithm

- To understand the implementation of selection sort algorithm, consider an unsorted list of numbers stored in an array.



Implementing Selection Sort Algorithm (Contd.)

- Let us sort this unsorted list.

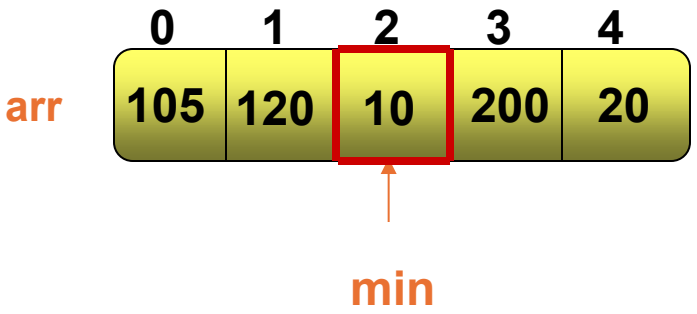


Implementing Selection Sort Algorithm (Contd.)

Pass 1

n = 5

- Search the minimum value in the array, arr[0] to arr[n – 1].

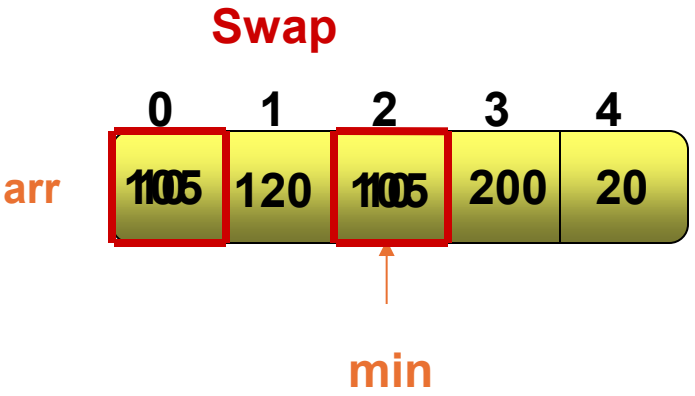


Implementing Selection Sort Algorithm (Contd.)

Pass 1

n = 5

- Search the minimum value in the array, arr[0] to arr[n – 1].
- Swap the minimum value with the value at index 0.

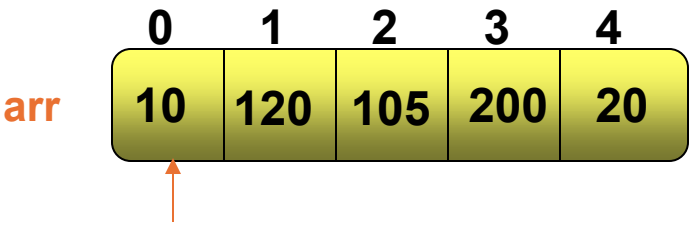


Implementing Selection Sort Algorithm (Contd.)

Pass 1

n = 5

- Search the minimum value in the array, arr[0] to arr[n – 1].
- Swap the minimum value with the value at index 0.



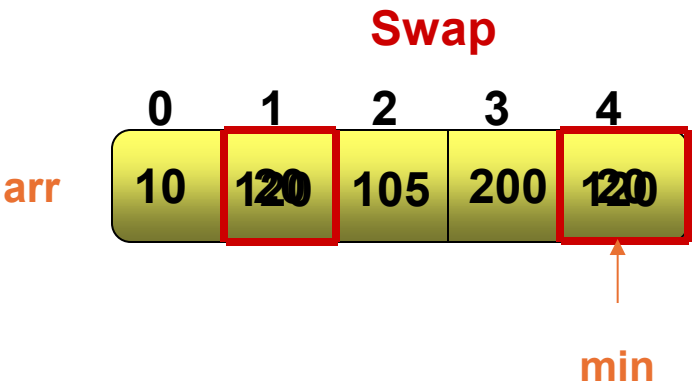
The smallest value is placed at its correct location after Pass 1

Implementing Selection Sort Algorithm (Contd.)

Pass 2

n = 5

- Search the minimum value in the array, arr[1] to arr[n – 1].
- Swap the minimum value with the value at index 1.

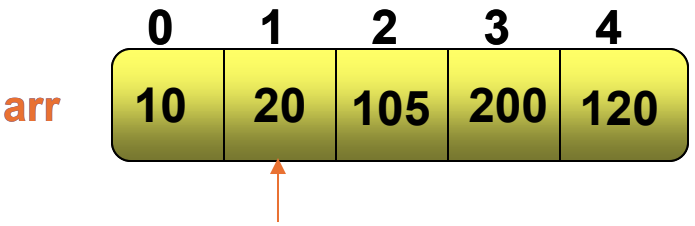


Implementing Selection Sort Algorithm (Contd.)

Pass 2

n = 5

- Search the minimum value in the array, arr[1] to arr[n – 1].
- Swap the minimum value with the value at index 1.



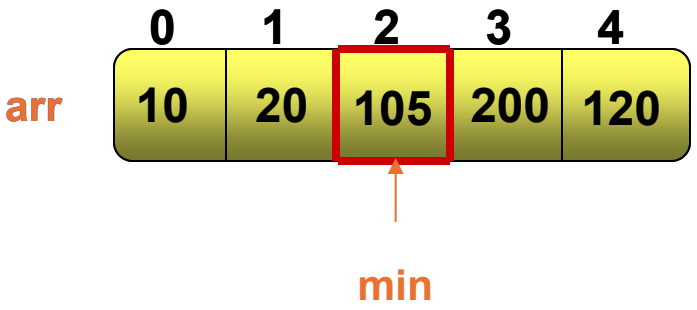
The second smallest value is placed at its correct location after Pass 2

Implementing Selection Sort Algorithm (Contd.)

Pass 3

n = 5

- Search the minimum value in the array, arr[2] to arr[n – 1].
- Swap the minimum value with the value at index 2.

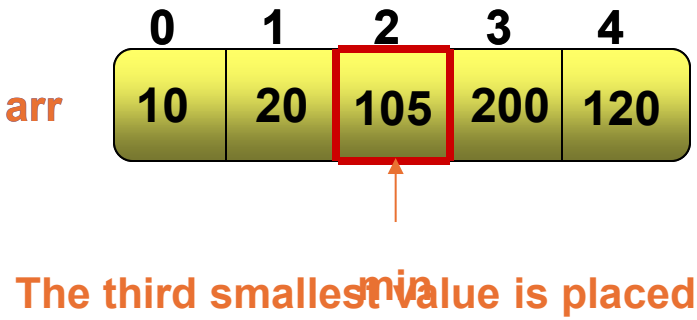


Implementing Selection Sort Algorithm (Contd.)

Pass 3

n = 5

- Search the minimum value in the array, arr[2] to arr[n – 1].
- Swap the minimum value with the value at index 2.



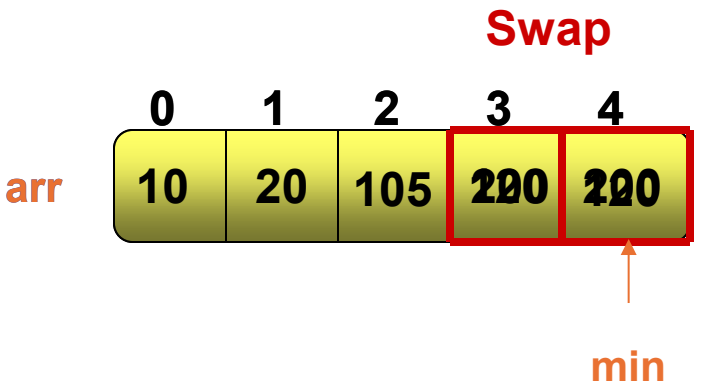
The third smallest value is placed at its correct location after Pass 3

Implementing Selection Sort Algorithm (Contd.)

Pass 4

n = 5

- Search the minimum value in the array, arr[3] to arr[n – 1].
- Swap the minimum value with the value at index 3.

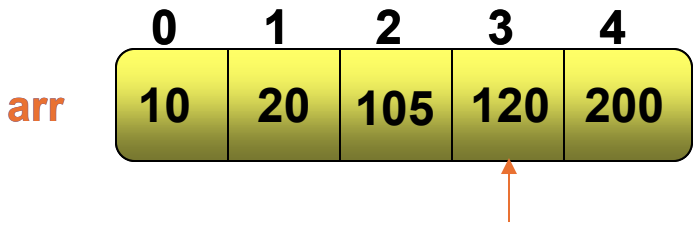


Implementing Selection Sort Algorithm (Contd.)

Pass 4

n = 5

- Search the minimum value in the array, arr[3] to arr[n – 1].
- Swap the minimum value with the value at index 3.



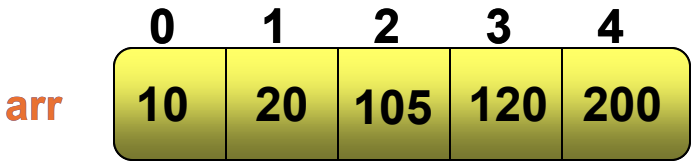
The fourth smallest value is placed at its correct location after Pass 4

Implementing Selection Sort Algorithm (Contd.)

Pass 4

n = 5

- The list is now sorted.



Selection Sort

```
procedure SelectionSort( int a[], int n )
 1. for (int i = 0; i < n - 1; i++)
  2.   int min = i;
  3. for (int j = i + 1; j < n; j++)
        3.1 if (a[min] > a[j])
              3.1.1 min = j;
        3.2 endif
   4. end for
    5. swap(a[i],a[min])
end procedure
```

Iterations – n-1

Comparisons – n-1 for 1st iteration, n-2 for 2nd iteration,.....1 comparison for (n-1) iteration

Total comparisons – 1+2+3+.....+(n-1)= n*(n+1)/2

Time Complexity - $O(n^2)$

Selection Sort

```
procedure SelectionSort( int A[], int n )

  1. for i = 0 to n-1
   2. for j = i+1 to n-1
              2.1 if list[i] > list[j] then          /* compare the adjacent
elements */ n-1
                    2.1.1 swap( list[i], list[j] )     /* swap them */
          2.2 end if
    3. end for
   4. end for
end procedure
```

Iterations – n-1
Comparisons – n-1 for 1st iteration, n-2 for 2nd iteration,.....1comparison for (n-1) comparison
Total comparisons – 1+2+3+.....+(n-1)= n*(n+1)/2

Time Complexity - $O(n^2)$

Selection sort

- Selection sort is internal sort method
- It is not stable algorithm
- Time Complexity (Best): $O(n^2)$
- Time Complexity (Average): $O(n^2)$
- Time Complexity (Worst): $O(n^2)$
- Space Complexity: O(1) – in-place sort

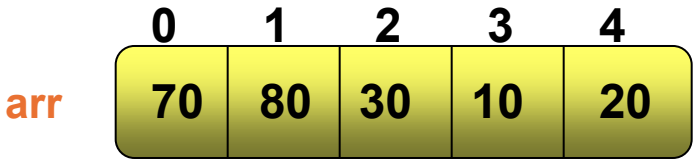
Insertion Sort

- Sorts a set of values by inserting values into an existing sorted array

- Steps:
  - Start with the second element as the first element is assumed to be sorted.
  - Compare the second element with the first if the second is smaller then swap them.
  - Move to the third element, compare it with the first two, and put it in its correct position
  - Repeat until the entire array is sorted.

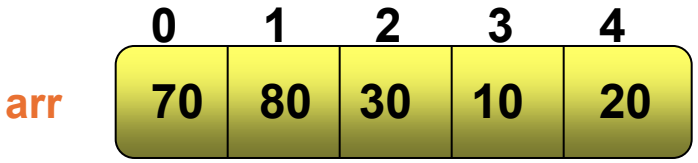
Implementing Insertion Sort Algorithm

- To understand the implementation of insertion sort algorithm, consider an unsorted list of numbers stored in an array.



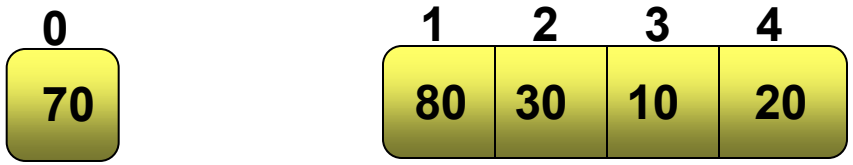
Implementing Insertion Sort Algorithm (Contd.)

- To sort this list by using the insertion sort algorithm:
  - You need to divide the list into two sublists, sorted and unsorted.



Implementing Insertion Sort Algorithm (Contd.)

- To sort this list by using the insertion sort algorithm:
  - You need to divide the list into two sublists, sorted and unsorted.
  - Initially, the sorted list has the first element and the unsorted list has the remaining 4 elements.



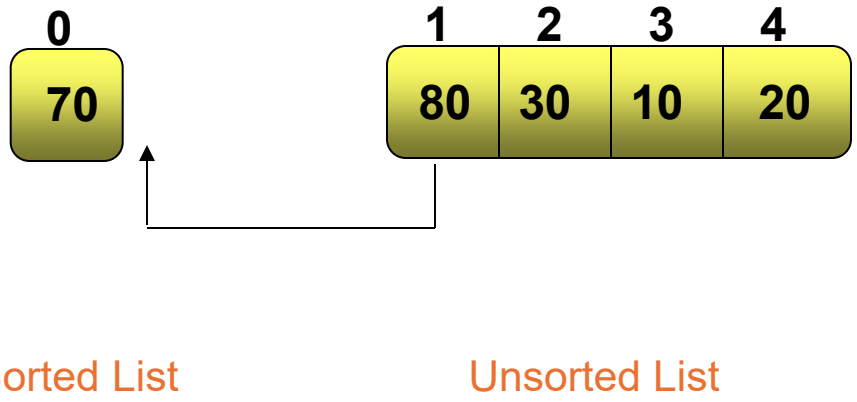
Sorted List

Unsorted List

Implementing Insertion Sort Algorithm (Contd.)

Pass 1

- Place the first element from the unsorted list at its correct position in the sorted list.



Implementing Insertion Sort Algorithm (Contd.)

Pass 1

- Place the first element from the unsorted list at its correct position in the sorted list.



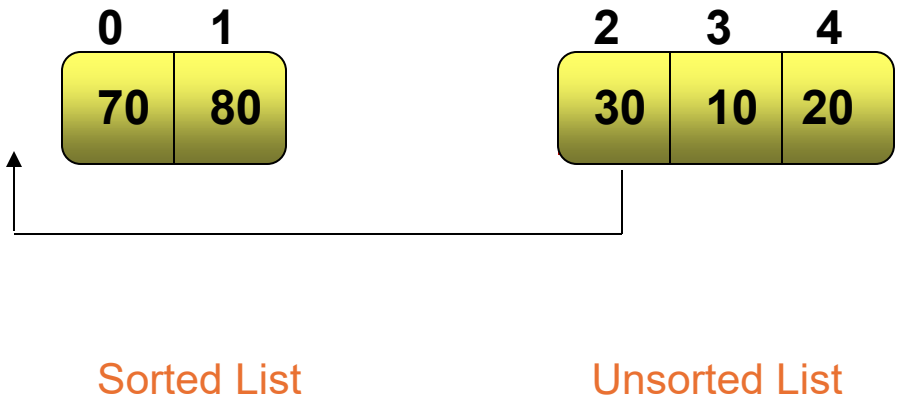
Sorted List

Unsorted List

Implementing Insertion Sort Algorithm (Contd.)

Pass 2

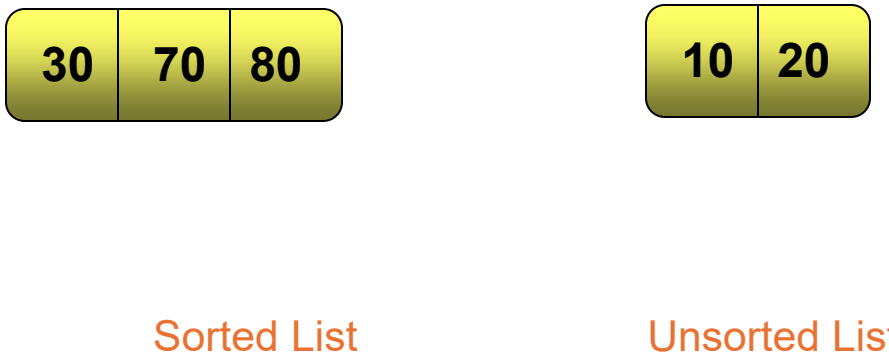
- Place the first element from the unsorted list at its correct position in the sorted list.



Implementing Insertion Sort Algorithm (Contd.)

Pass 2

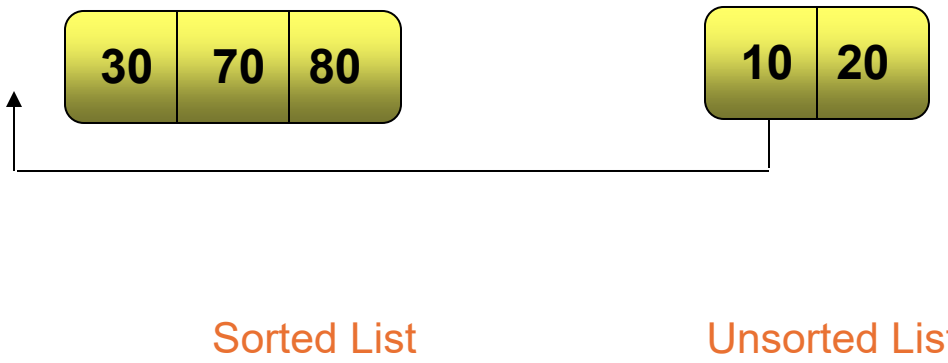
- Place the first element from the unsorted list at its correct position in the sorted list.



Implementing Insertion Sort Algorithm (Contd.)

Pass 3

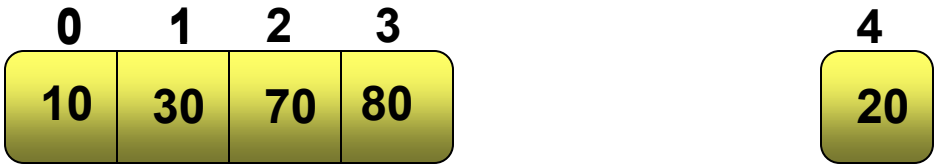
- Place the first element from the unsorted list at its correct position in the sorted list.



Implementing Insertion Sort Algorithm (Contd.)

Pass 3

- Place the first element from the unsorted list at its correct position in the sorted list.



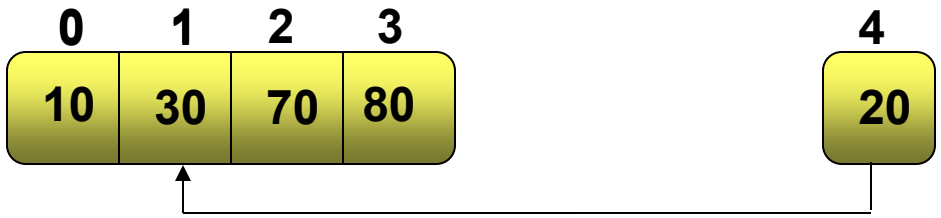
Sorted List

Unsorted List

Implementing Insertion Sort Algorithm (Contd.)

Pass 4

- Place the first element from the unsorted list at its correct position in the sorted list.



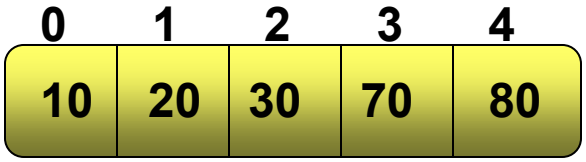
Sorted List

Unsorted List

Implementing Insertion Sort Algorithm (Contd.)

Pass 4

- Place the first element from the unsorted list at its correct position in the sorted list.



Sorted List

Unsorted List

Insertion Sort

Procedure insertionSort(int arr[], int n)

1. for (i = 1; i < n; i++)
2. key = arr[i];
3. j = i - 1;
4. while (j >= 0 && arr[j] > key) /* Move elements of arr[0..i-1], that are greater than key, to one position ahead of their current position */

    4.1 arr[j + 1] = arr[j];

    4.2 j = j - 1;
5. End while
6. arr[j + 1] = key;
7. End for

Iterations – n-1
Comparisons – n-1 for 1st iteration, n-2 for 2nd iteration,.....1comparison for (n-1) comparison
Total comparisons – 1+2+3+.....+(n-1)= n*(n+1)/2

Time Complexity - $O(n^2)$

Quick Sort

- Also called as partition exchange sort
- It works by partitioning an array into two parts, then sorting the parts independently.
- Steps:
  - Choose a Pivot: Select an element from the array as the pivot. The choice of pivot can vary (e.g., first element, last element, random element, or median).
  - Partition the Array: Re arrange the array around the pivot. After partitioning, all elements smaller than the pivot will be on its left, and all elements greater than the pivot will be on its right.
  - Recursively Call: Recursively apply the same process to the two partitioned sub-arrays.
  - Base Case: The recursion stops when there is only one element left in the sub-array, as a single element is already sorted.

```
void quicksort(int a[], int left, int right, int n)
{
   if(left < right)
   {
      int i = left;
      int j = right + 1;
      int pivot = a[left];
      int temp;

      // Partitioning
      while(i < j)
      {
         do
         {
            i++;
         } while(i <= right && a[i] < pivot);
do
         {
            j--;
         } while(a[j] > pivot);

         if(i < j)
         {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
         }
      }
       // Place pivot in its correct position
      temp = a[left];
      a[left] = a[j];
      a[j] = temp;
      // Recursive calls
      quicksort(a, left, j - 1, n);
      quicksort(a, j + 1, right, n);
   }
```

Recurrence relation for Quick Sort

- Average case

$$T(n) = T(n/2) + T(n/2) + O(n)$$

$$\Rightarrow T(n) = 2T(n/2) + O(n)$$

- Worst Case

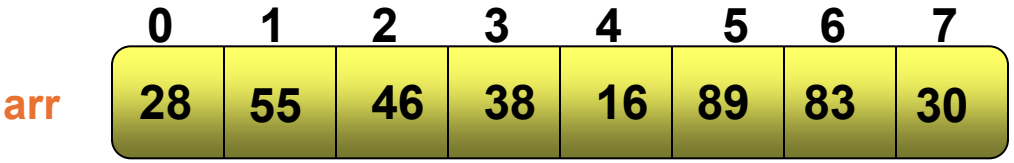
$$T(n) = T(n-1) + O(n)$$

$$T(n) = O(n) + O(n-1) + \cdots + O(1)$$

$$\Rightarrow T(n) = O(n^2)$$

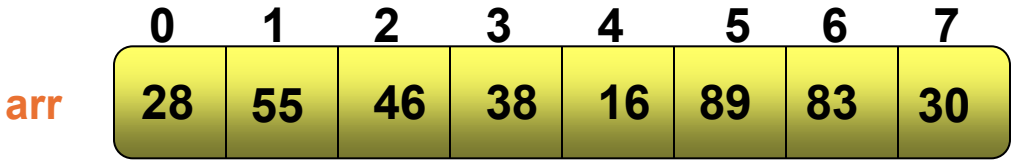
Implementing Quick Sort Algorithm (Contd.)

- To understand the implementation of quick sort algorithm, consider an unsorted list of numbers stored in an array.



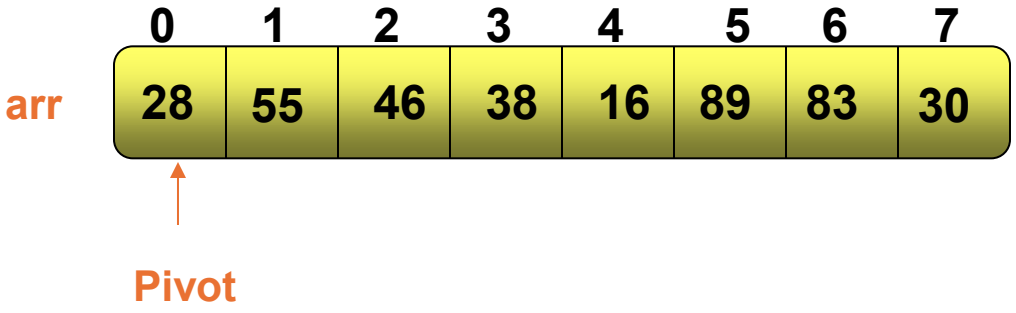
Implementing Quick Sort Algorithm (Contd.)

- Let us sort this unsorted list.



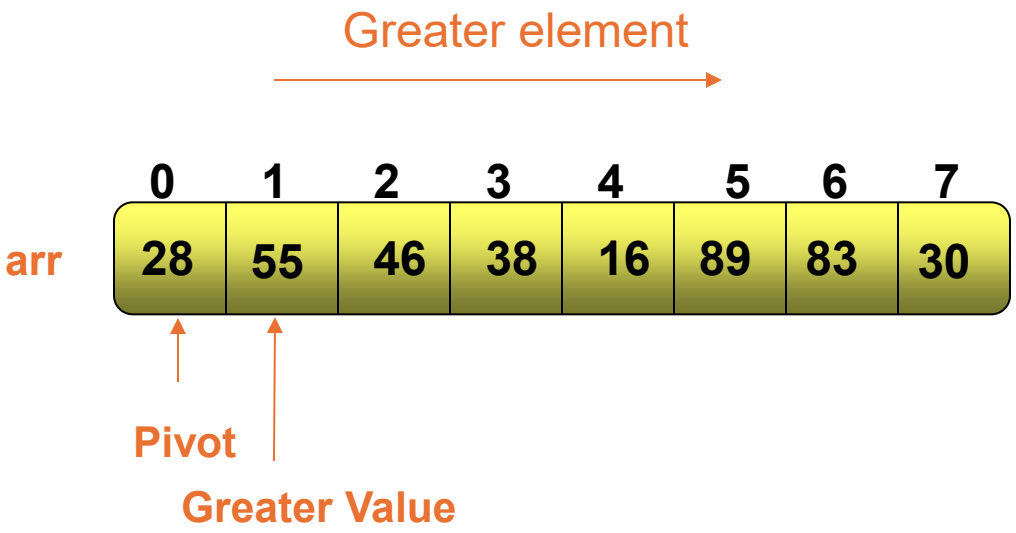
Implementing Quick Sort Algorithm (Contd.)

- Select a Pivot.



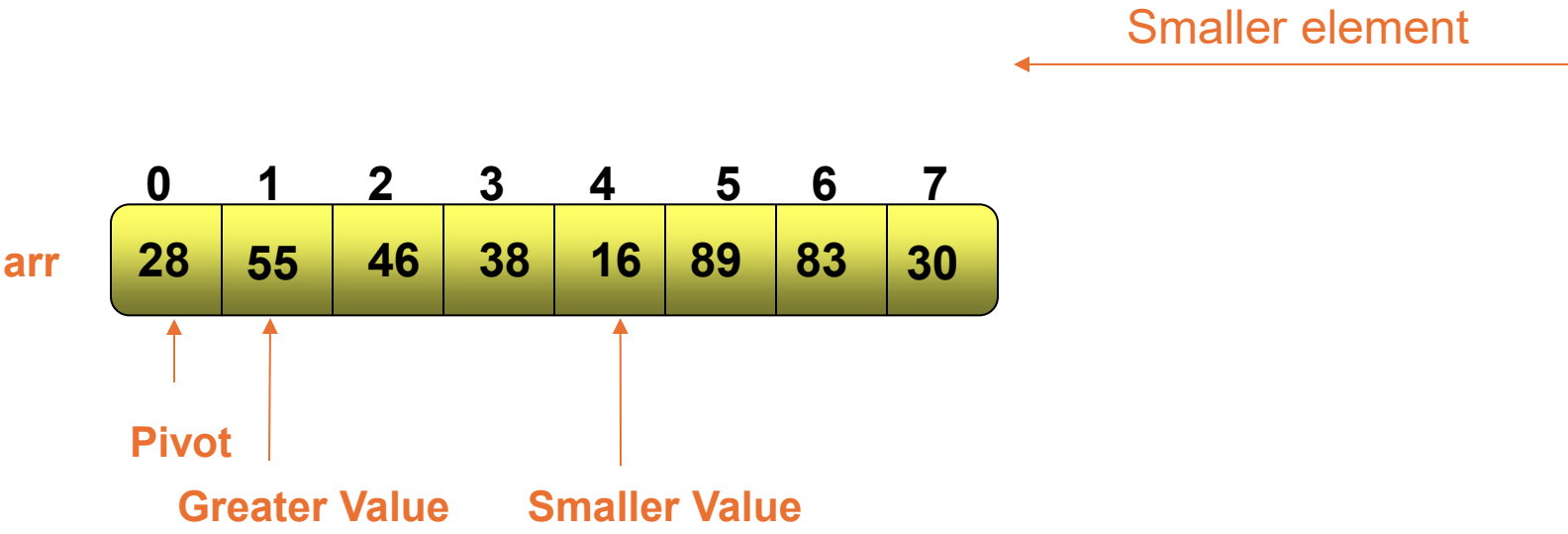
Implementing Quick Sort Algorithm (Contd.)

- Start from the left end of the list (at index 1).
- Move in the left to right direction.
- Search for the first element that is greater than the pivot value.



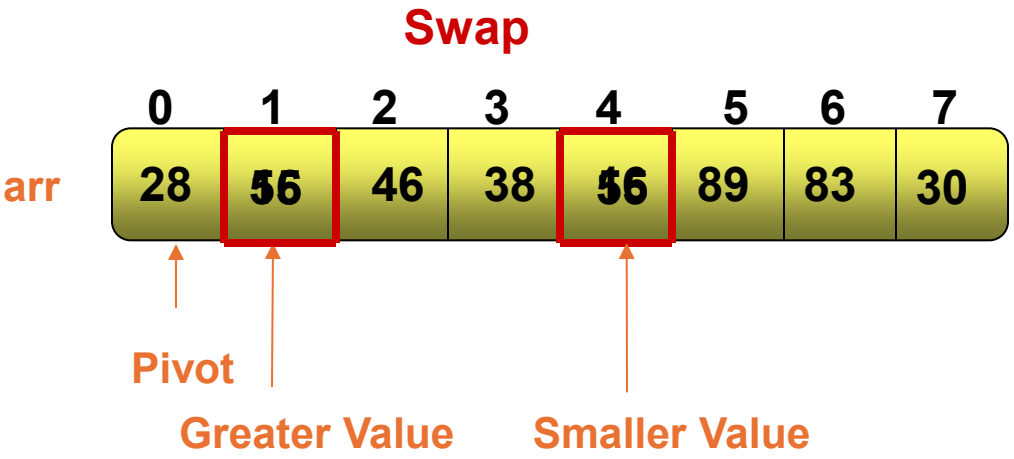
Implementing Quick Sort Algorithm (Contd.)

- Start from the right end of the list.
- Move in the right to left direction.
- Search for the first element that is smaller than or equal to the pivot value.



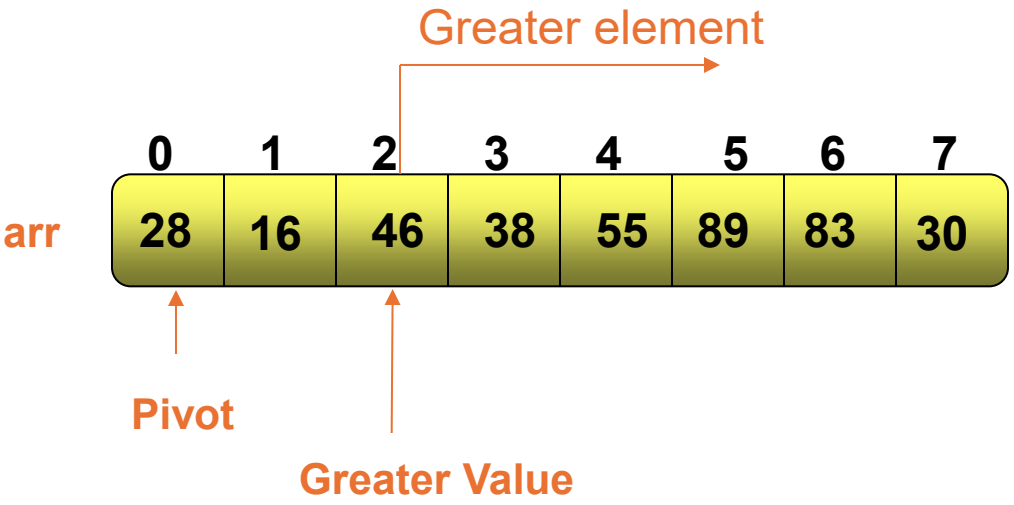
Implementing Quick Sort Algorithm (Contd.)

- Interchange the greater value with smaller value.



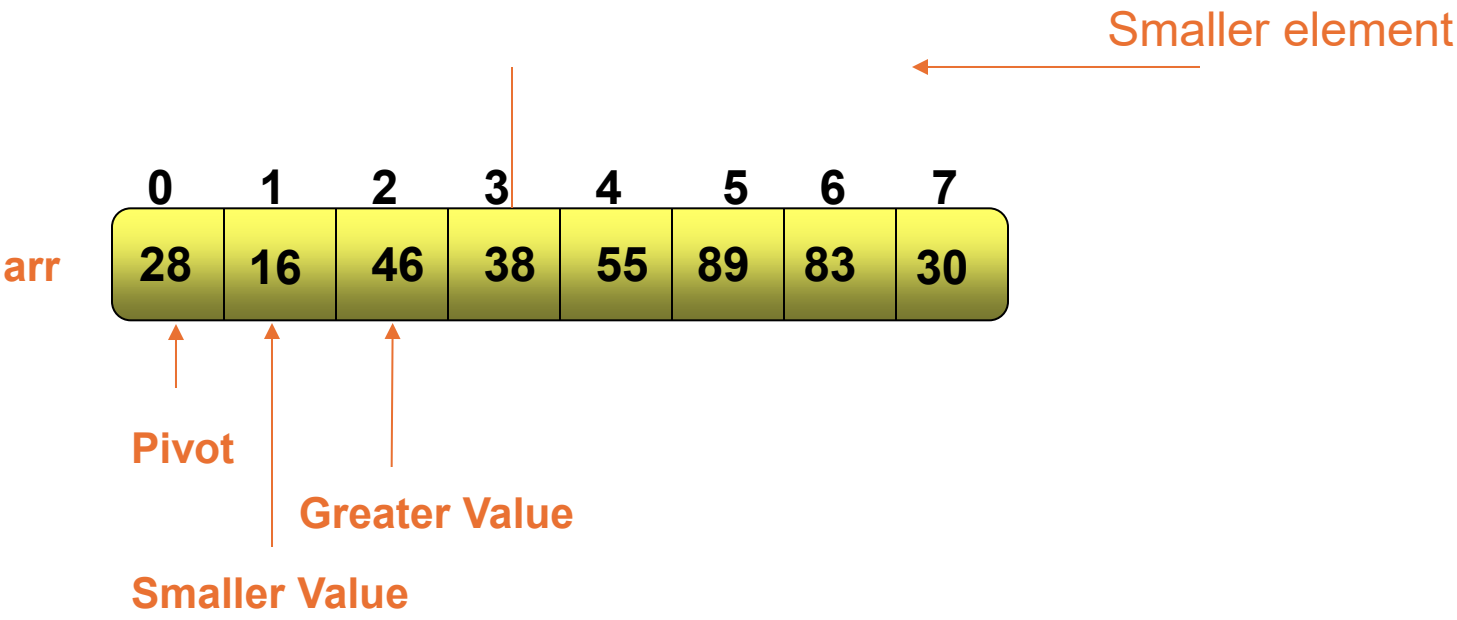
Implementing Quick Sort Algorithm (Contd.)

- Continue the search for an element greater than the pivot.
- Start from arr[2] and move in the left to right direction.
- Search for the first element that is greater than the pivot value.



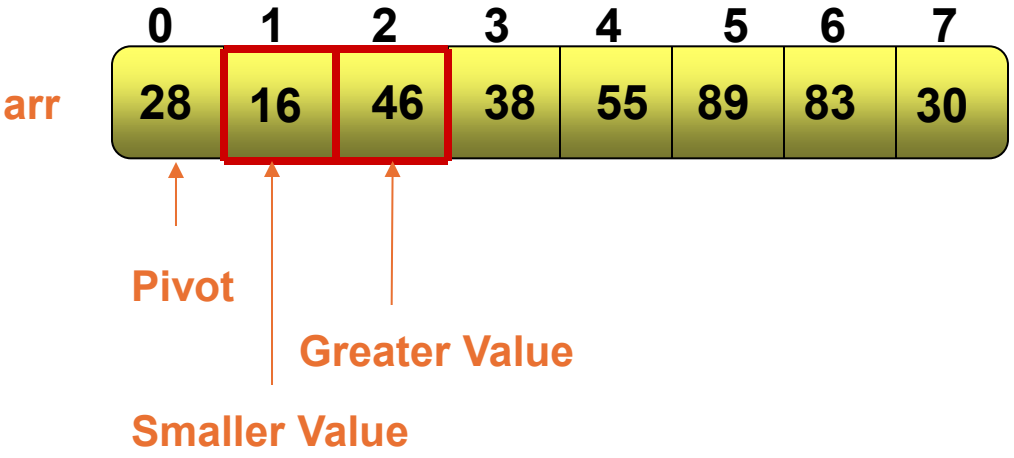
Implementing Quick Sort Algorithm (Contd.)

- Continue the search for an element smaller than the pivot.
- Start from arr[3] and move in the right to left direction.
- Search for the first element that is smaller than or equal to the pivot value.



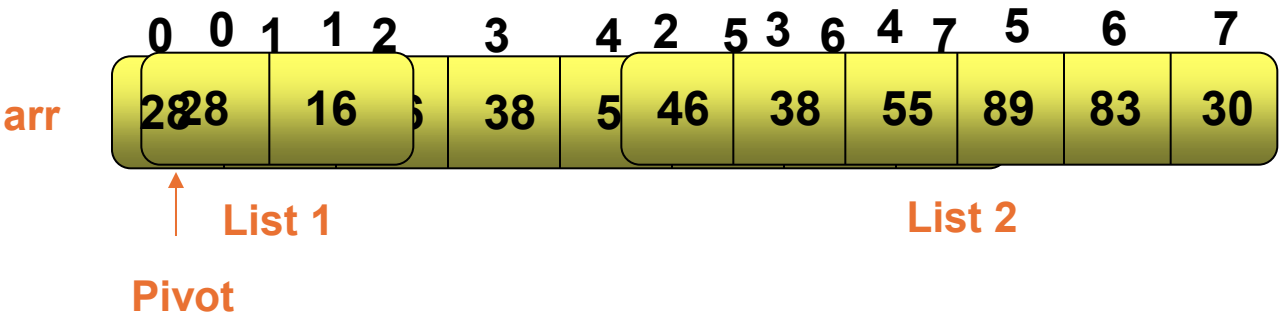
Implementing Quick Sort Algorithm (Contd.)

- The smaller value is on the left hand side of the greater value.
- Values remain same.



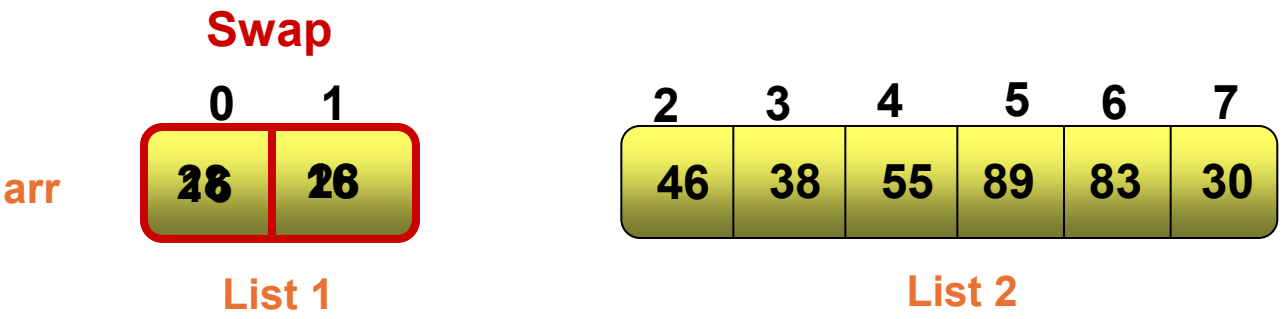
Implementing Quick Sort Algorithm (Contd.)

- List is now partitioned into two sublists.
- List 1 contains all values less than or equal to the pivot.
- List 2 contains all the values greater than the pivot.



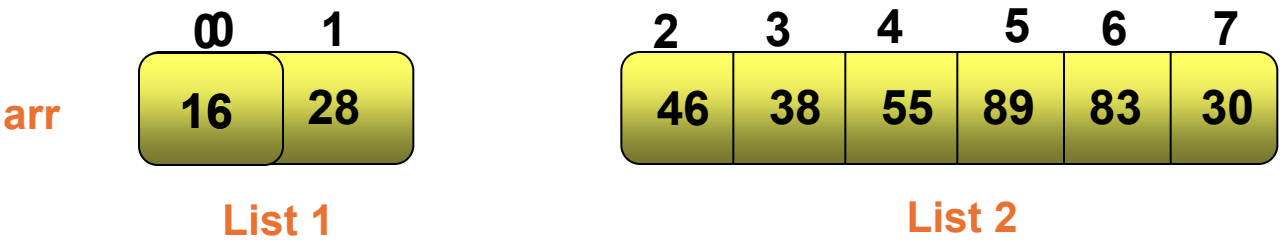
Implementing Quick Sort Algorithm (Contd.)

- Replace the pivot value with the last element of List 1.
- The pivot value, 28 is now placed at its correct position in the list.



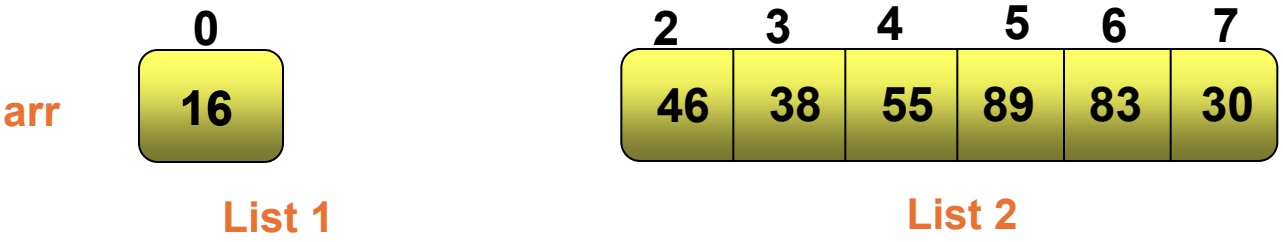
Implementing Quick Sort Algorithm (Contd.)

- Truncate the last element, that is, pivot from List 1.



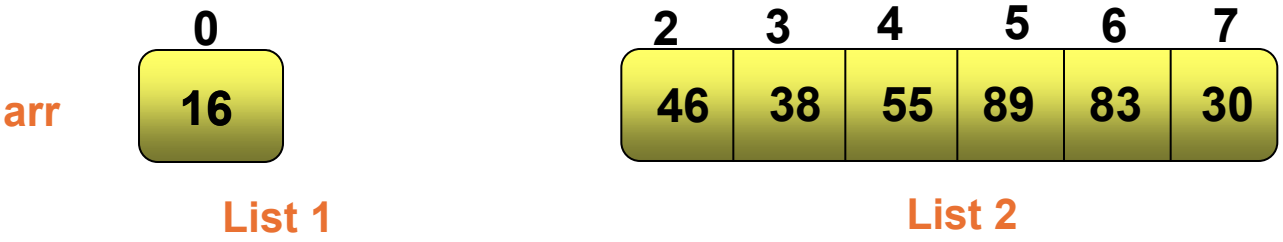
Implementing Quick Sort Algorithm (Contd.)

- List 1 has only one element.
- Therefore, no sorting required.



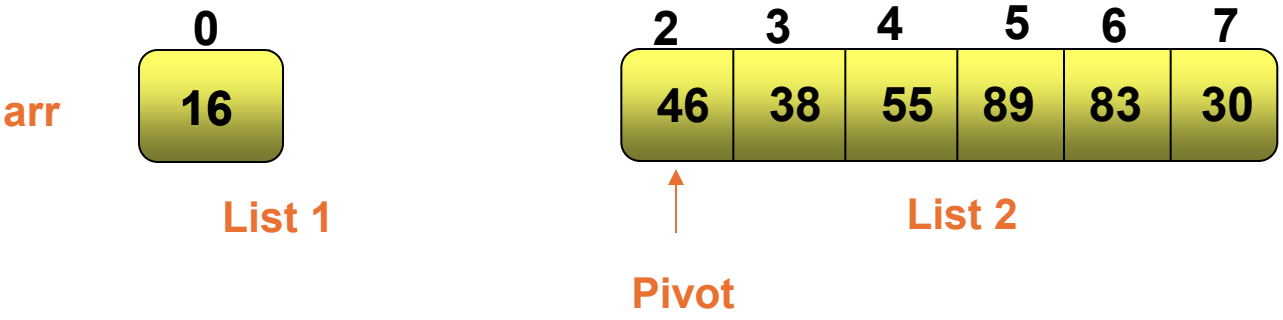
Implementing Quick Sort Algorithm (Contd.)

- Sort the second list, List 2.



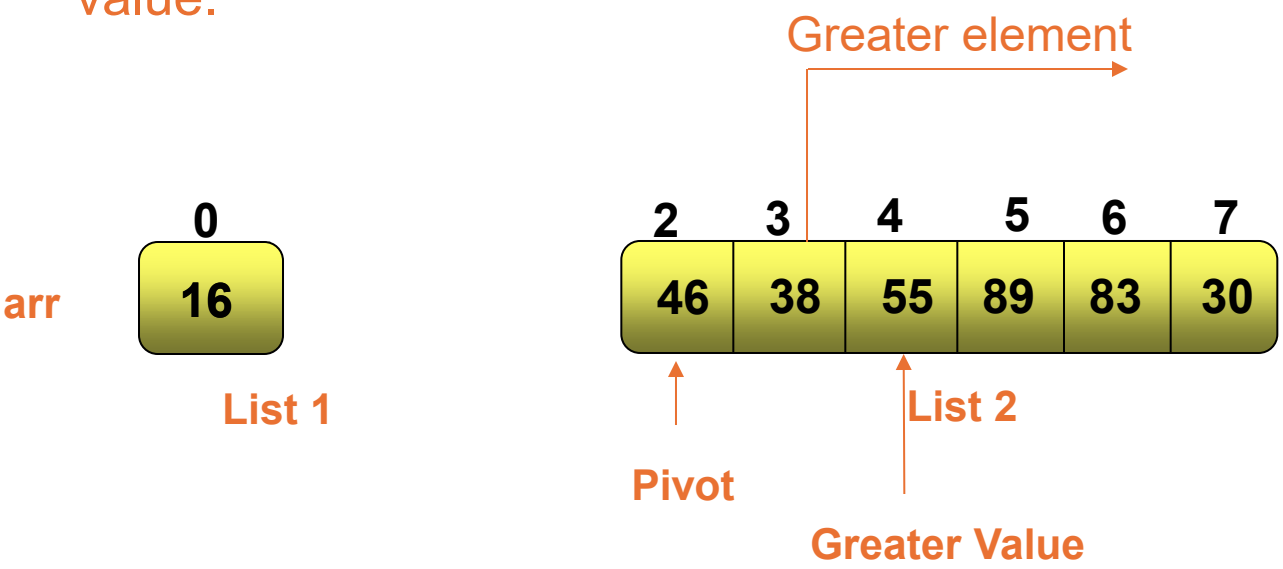
Implementing Quick Sort Algorithm (Contd.)

- Select a pivot.
- The pivot in this case will be arr[2], that is, 46.



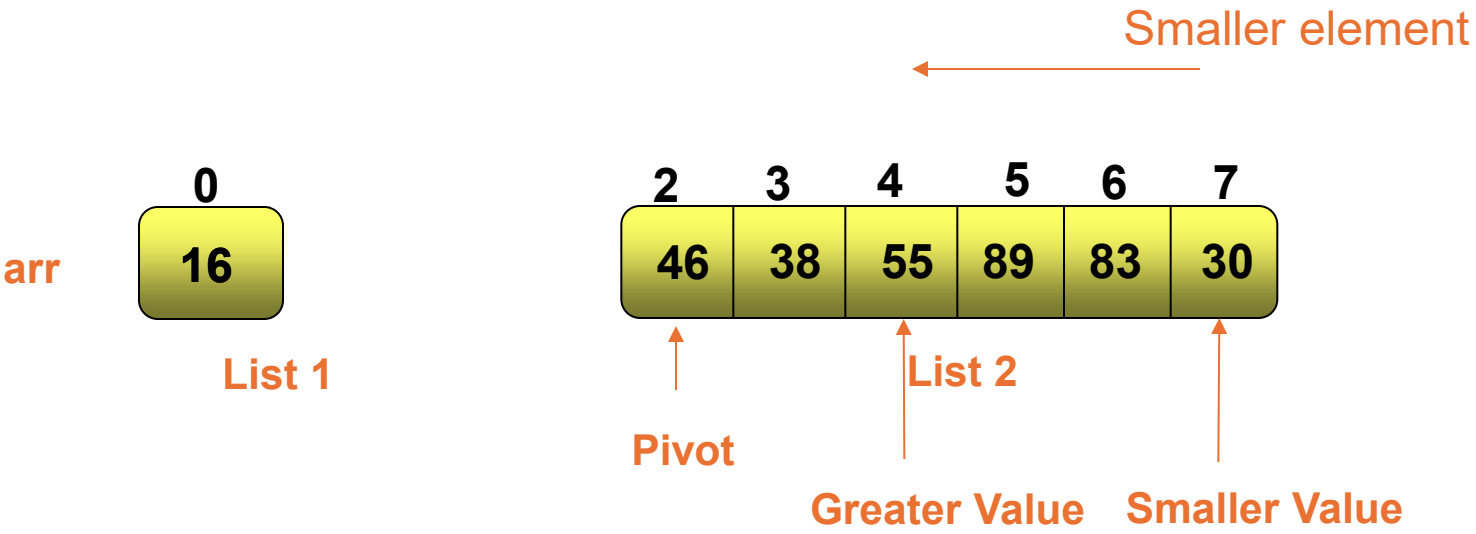
Implementing Quick Sort Algorithm (Contd.)

- Start from the left end of the list (at index 3).
- Move in the left to right direction.
- Search for the first element that is greater than the pivot value.



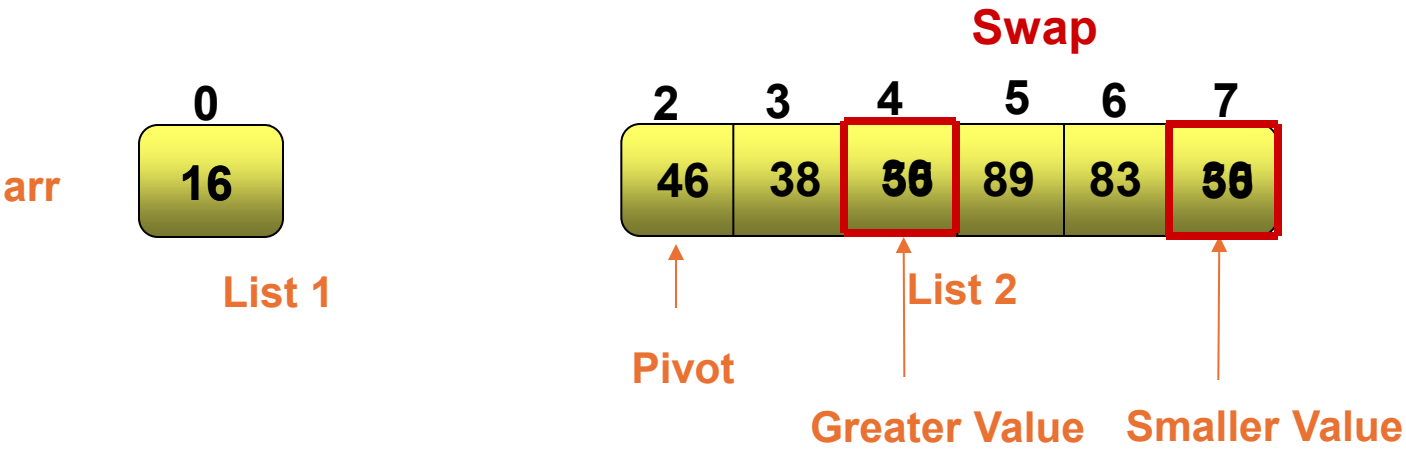
Implementing Quick Sort Algorithm (Contd.)

- Start from the right end of the list (at index 7).
- Move in the right to left direction.
- Search for the first element that is smaller than or equal to the pivot value.



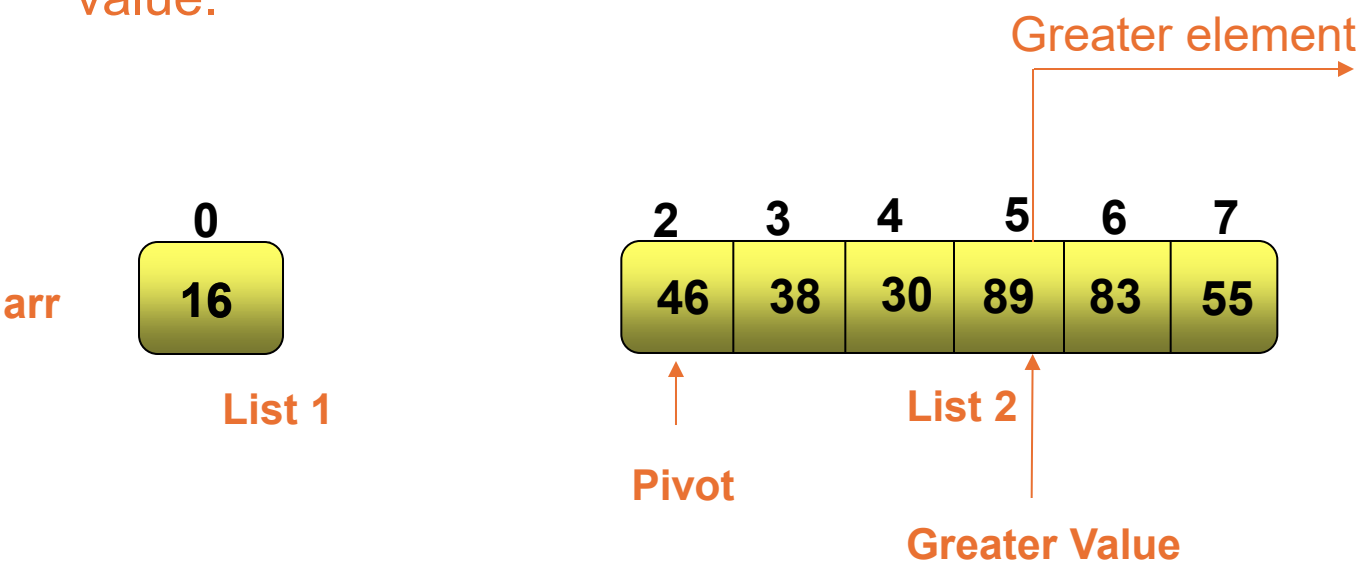
Implementing Quick Sort Algorithm (Contd.)

- Interchange the greater value with smaller value.



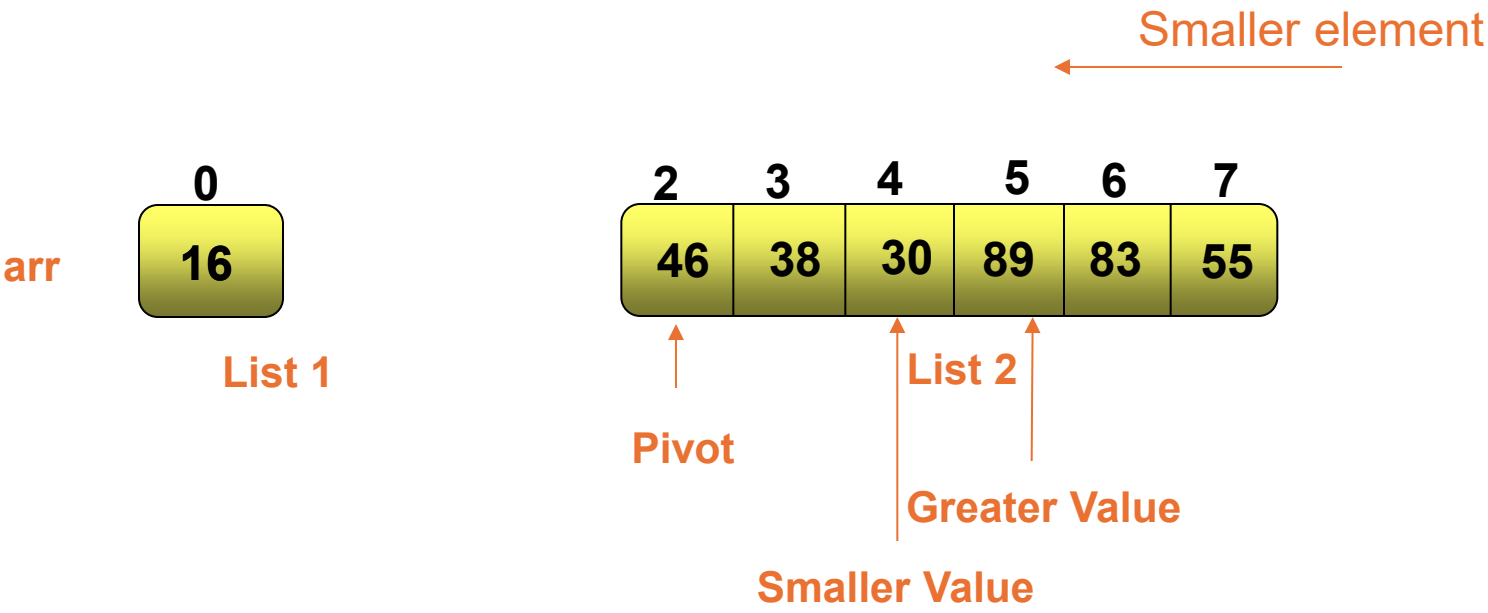
Implementing Quick Sort Algorithm (Contd.)

- Continue the search for an element greater than the pivot.
- Start from arr[5] and move in the left to right direction.
- Search for the first element that is greater than the pivot value.



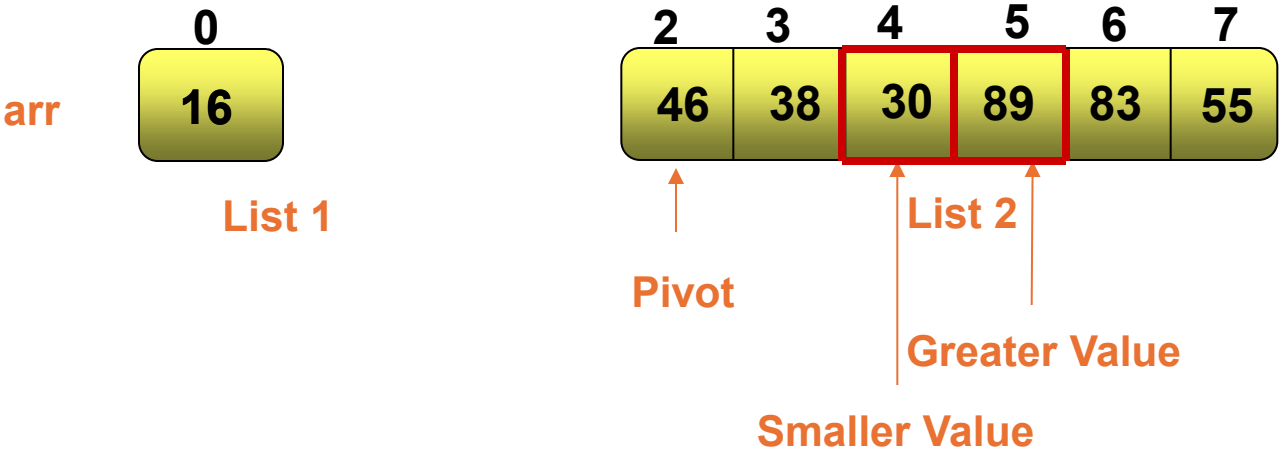
Implementing Quick Sort Algorithm (Contd.)

- Continue the search for an element smaller than the pivot.
- Start from arr[6] and move in the right to left direction.
- Search for the first element that is smaller than the pivot value.



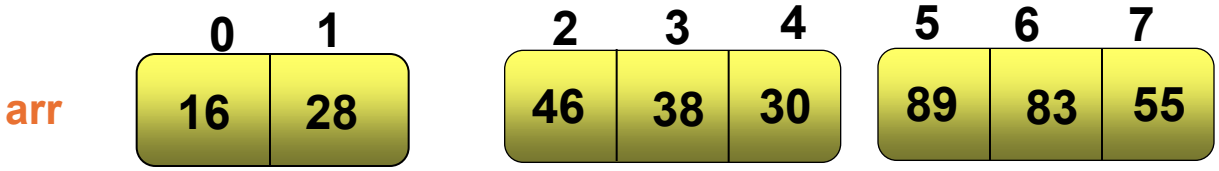
Implementing Quick Sort Algorithm (Contd.)

- The smaller value is on the left hand side of the greater value.
- Values remain same.



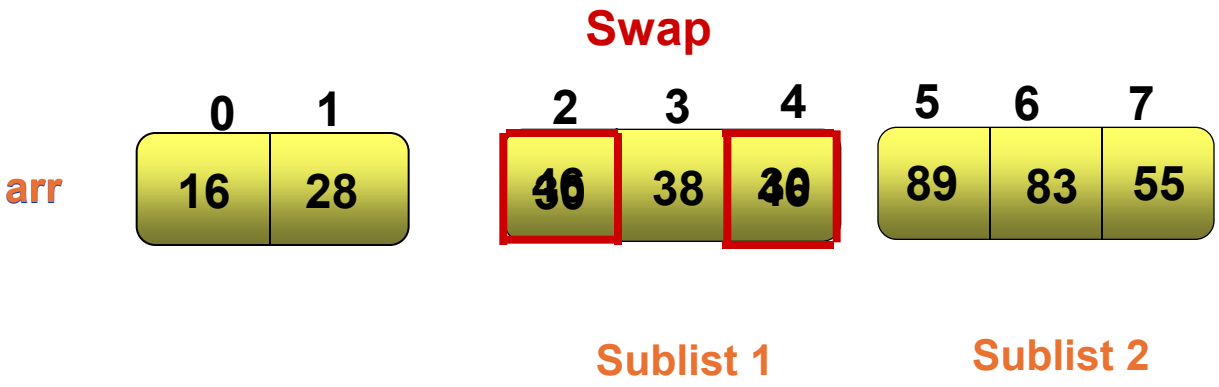
Implementing Quick Sort Algorithm (Contd.)

- Divide the list into two sublists.
- Sublist 1 contains all values less than or equal to the pivot.
- Sublist 2 contains all the values greater than the pivot.



Implementing Quick Sort Algorithm (Contd.)

- Replace the pivot value with the last element of Sublist 1.
- The pivot value, 46 is now placed at its correct position in the list.
- This process is repeated until all elements reach their correct position.

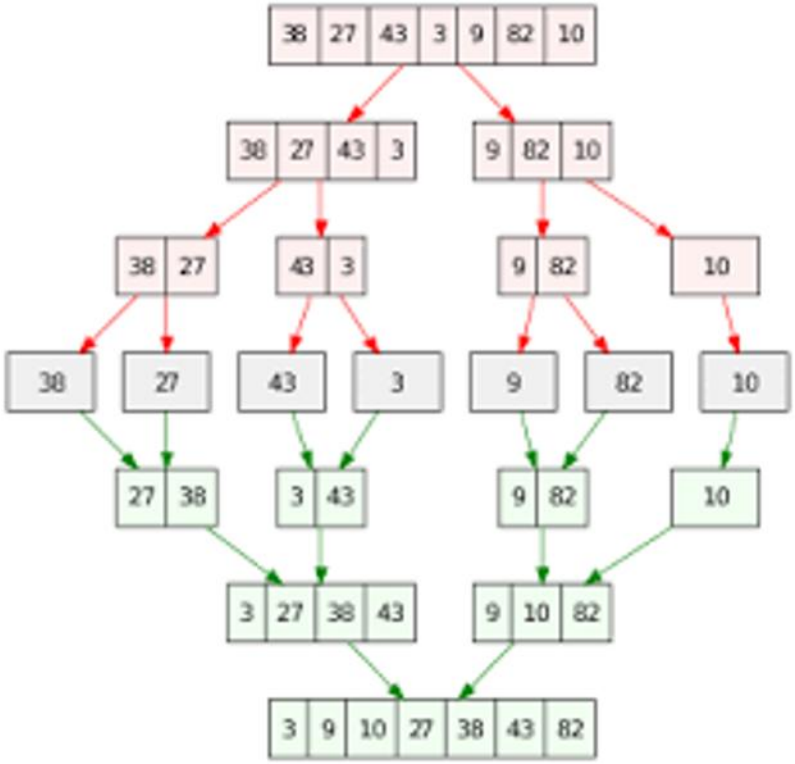


Merge Sort

- Is based on the divide and conquer approach
- Divides the list into two sublists of sizes as nearly equal as possible
- Sorts the two sublists separately by using merge sort
- Merges the sorted sublists into one single list
- 2 approach to implement
  - Top down approach
  - Bottom up approach

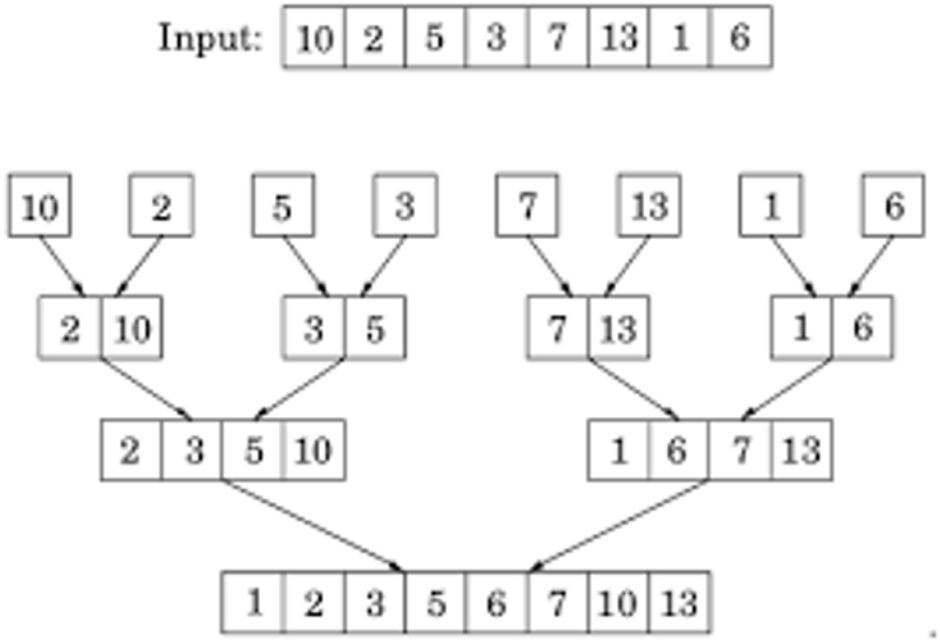
Merge Sort- Top down approach

- At start assume single array of n size
- Divide it in two subarrays of n/2 size each
- Continue division till subarray of size 1
- Apply merging on 2 subarrays of size 1
- Repeat merging process till merging of 2 subarrays of n/2 size happens



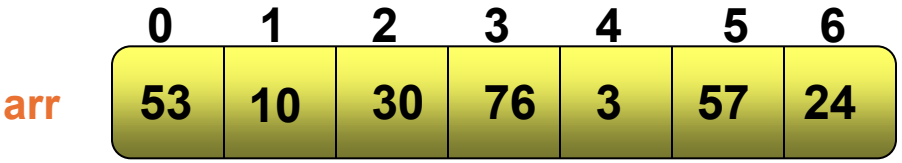
Merge Sort- Top down approach

- At start assume n arrays of size ‘1’
- Apply merging on 2 arrays of size 1
- Repeat merging process till merging of 2 subarrays of n/2 size happens



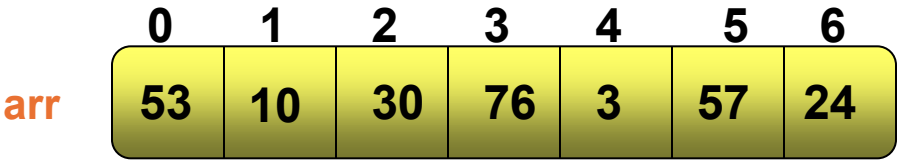
Implementing Merge Sort Algorithm

- To understand the implementation of merge sort algorithm, consider an unsorted list of numbers stored in an array.



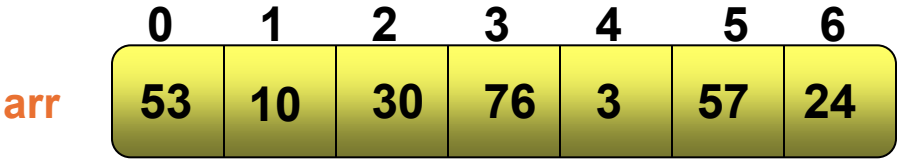
Implementing Merge Sort Algorithm (Contd.)

- Let us sort this unsorted list.



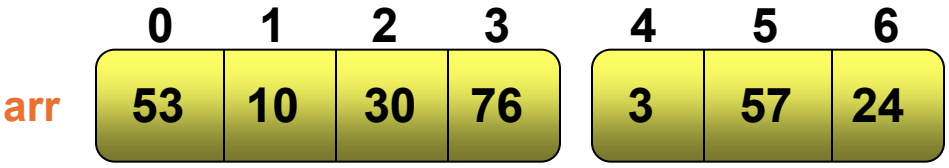
Implementing Merge Sort Algorithm (Contd.)

- The first step to sort data by using merge sort is to split the list into two parts.



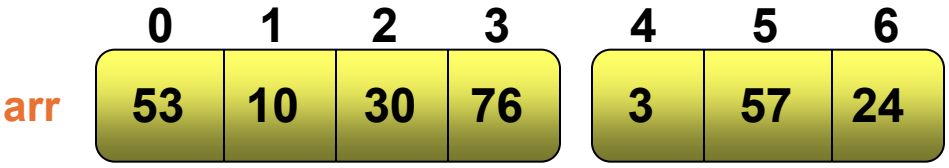
Implementing Merge Sort Algorithm (Contd.)

- The first step to sort data by using merge sort is to split the list into two parts.



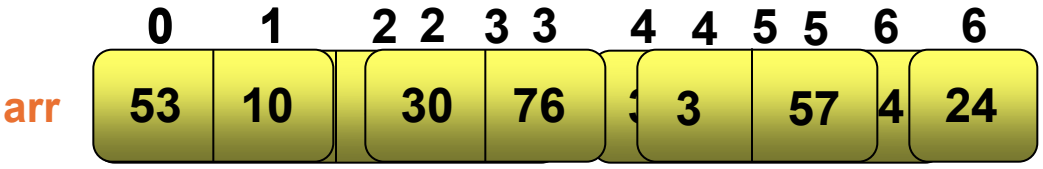
Implementing Merge Sort Algorithm (Contd.)

- The list has odd number of elements, therefore, the left sublist is longer than the right sublist by one entry.



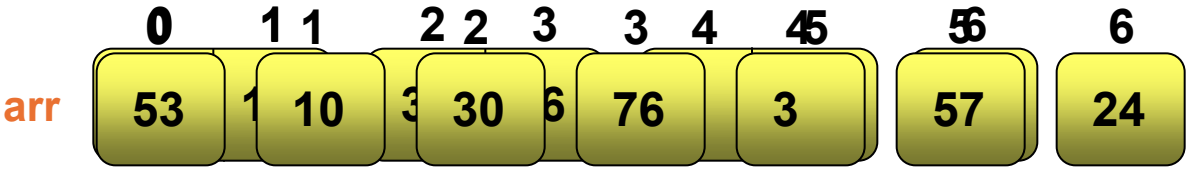
Implementing Merge Sort Algorithm (Contd.)

- Further divide the two sublists into nearly equal parts.



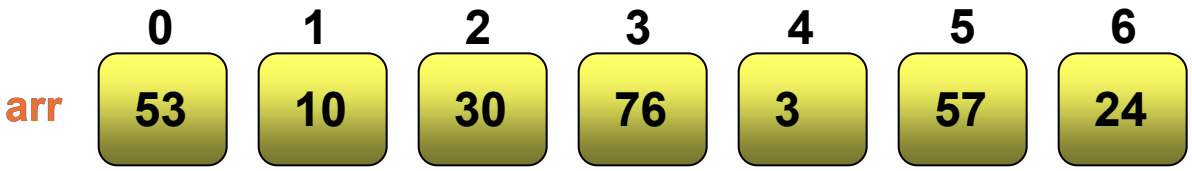
Implementing Merge Sort Algorithm (Contd.)

- Further divide the sublists.



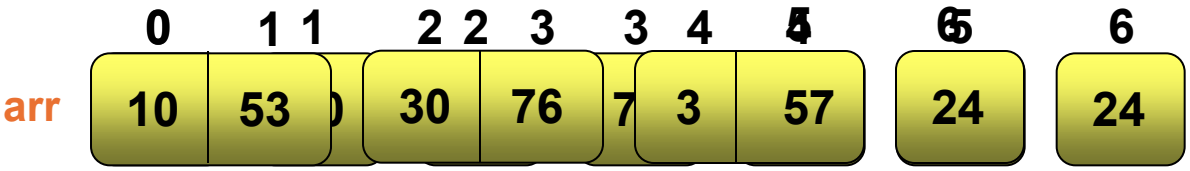
Implementing Merge Sort Algorithm (Contd.)

- There is a single element left in each sublist.
- Sublists with one element require no sorting.



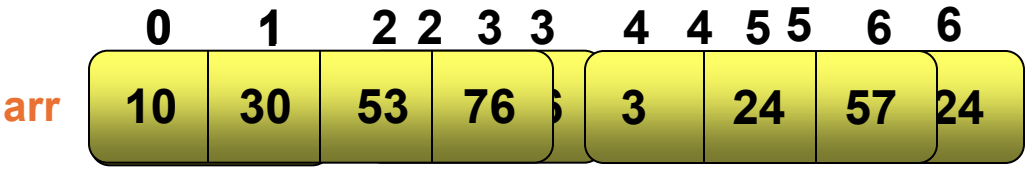
Implementing Merge Sort Algorithm (Contd.)

- Start merging the sublists to obtain a sorted list.



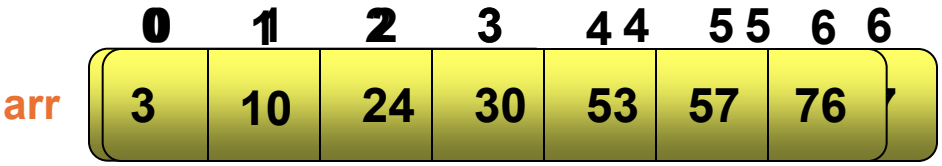
Implementing Merge Sort Algorithm (Contd.)

- Further merge the sublists.



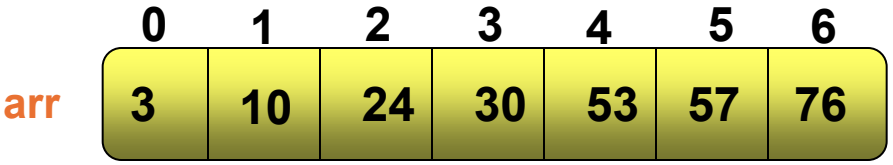
Implementing Merge Sort Algorithm (Contd.)

- Again, merge the sublists.



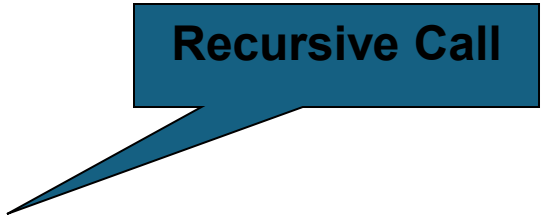
Implementing Merge Sort Algorithm (Contd.)

- The list is now sorted.



```
void Merge_Sort(int A[], int low, int high, int n)
{
   if (low >= high)
      return;

   int mid = (low + high) / 2;
   Merge_Sort(A, low, mid, n);
   Merge_Sort(A, mid + 1, high, n);
   Merge(A, low, mid, high, n);
}
```



```
void Merge(int A[], int low, int mid, int high, int n)
{
   int B[100];
   int i = low, j = mid + 1, k = low;

   while (i <= mid && j <= high)
   {
      if (A[i] <= A[j])
         B[k++] = A[i++];
      else
         B[k++] = A[j++];
   }

   while (i <= mid)
      B[k++] = A[i++];

   while (j <= high)
      B[k++] = A[j++];

   // Copy back to original array
   for (i = low; i <= high; i++)
      A[i] = B[i];

}
Call from main(): Merge_Sort(A, 0, n - 1, n);
```

At each level, we merge all elements — total of n comparisons across the entire level.

- For example: Top level: merge two n/2 arrays → total work = n
- Next level: merge four n/4 arrays → total work = n
- This happens at every level

Space Complexity O(n) extra space is used due to temporary arrays during merging. Merge Sort is not in-place.

Total time=number of levels × work per level=log2 (n)×n=O(nlogn)

Case	Time Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)

Comparison of Sorting Algorithms

Sorting Algorithm	Best Case Time	Average Case Time	Worst Case Time	Space Complexity	Stability	In-Place	Notes
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes	Simple but inefficient for large data.
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	Yes	Always makes n swaps; not adaptive.
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes	Efficient for small or nearly sorted data.
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(1)	No	Yes	Very fast in practice; uses divide and conquer.
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	No	Guarantees performance; preferred for linked lists.